

Static analysis and software verification

Lecture 9 - Abstract interpretation in LiSA

Vincenzo Arceri - University of Parma - vincenzo.arceri@unipr.it

LiSA, a Library for Static Analysis

- Open source library written in Java
- Created and maintained by the SSV group @ UniVE
- LiSA program is made up of CFGs (+ extra information)
- CFGs encode flow control in their structure
- CFG nodes are language-specific
- CFG nodes rewrite into symbolic expressions

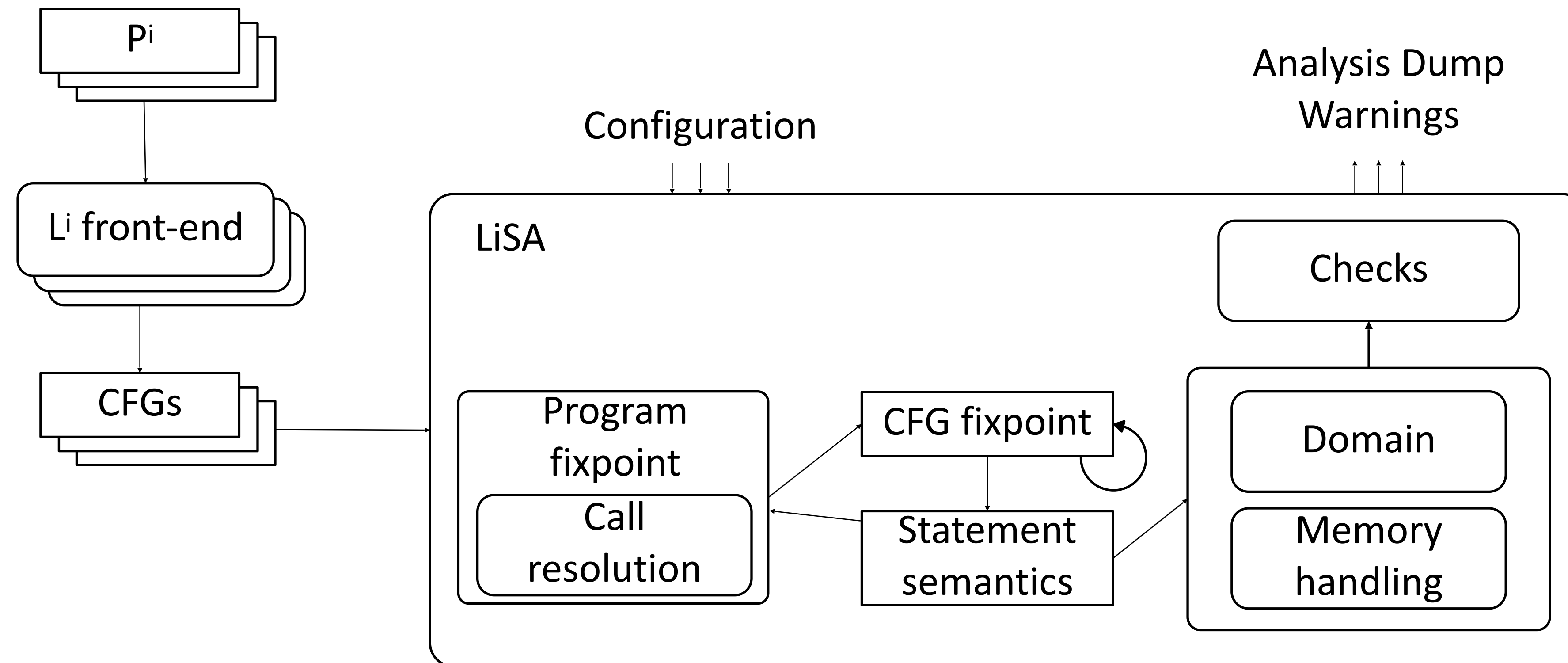
LiSA, a Library for Static Analysis

- Open source library written in Java
- Created and maintained by the SSV group @ UniVE
- LiSA program is made up of CFGs (+ extra information)
- CFGs encode flow control in their structure
- CFG nodes are language-specific
- CFG nodes rewrite into symbolic expressions

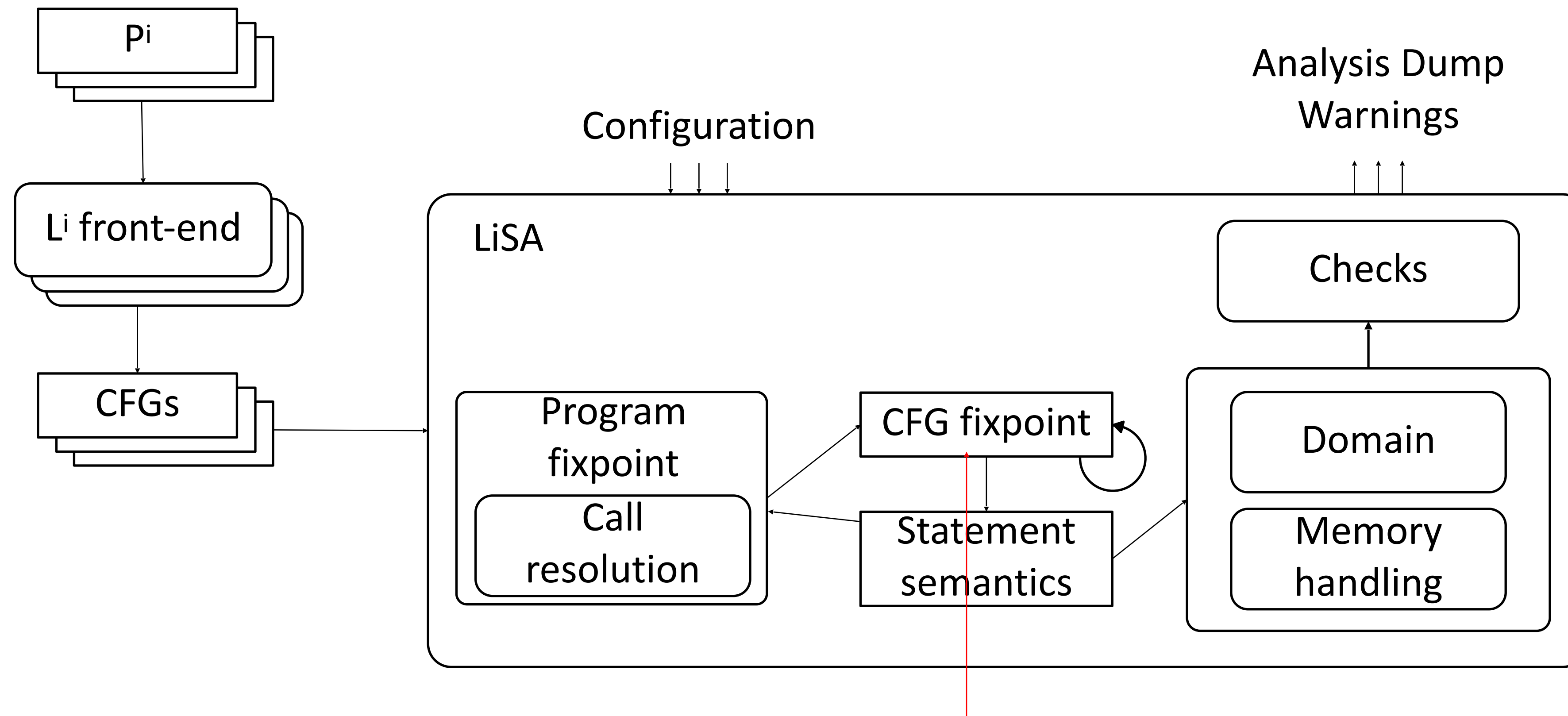
Syntax

Semantics

LiSA overview

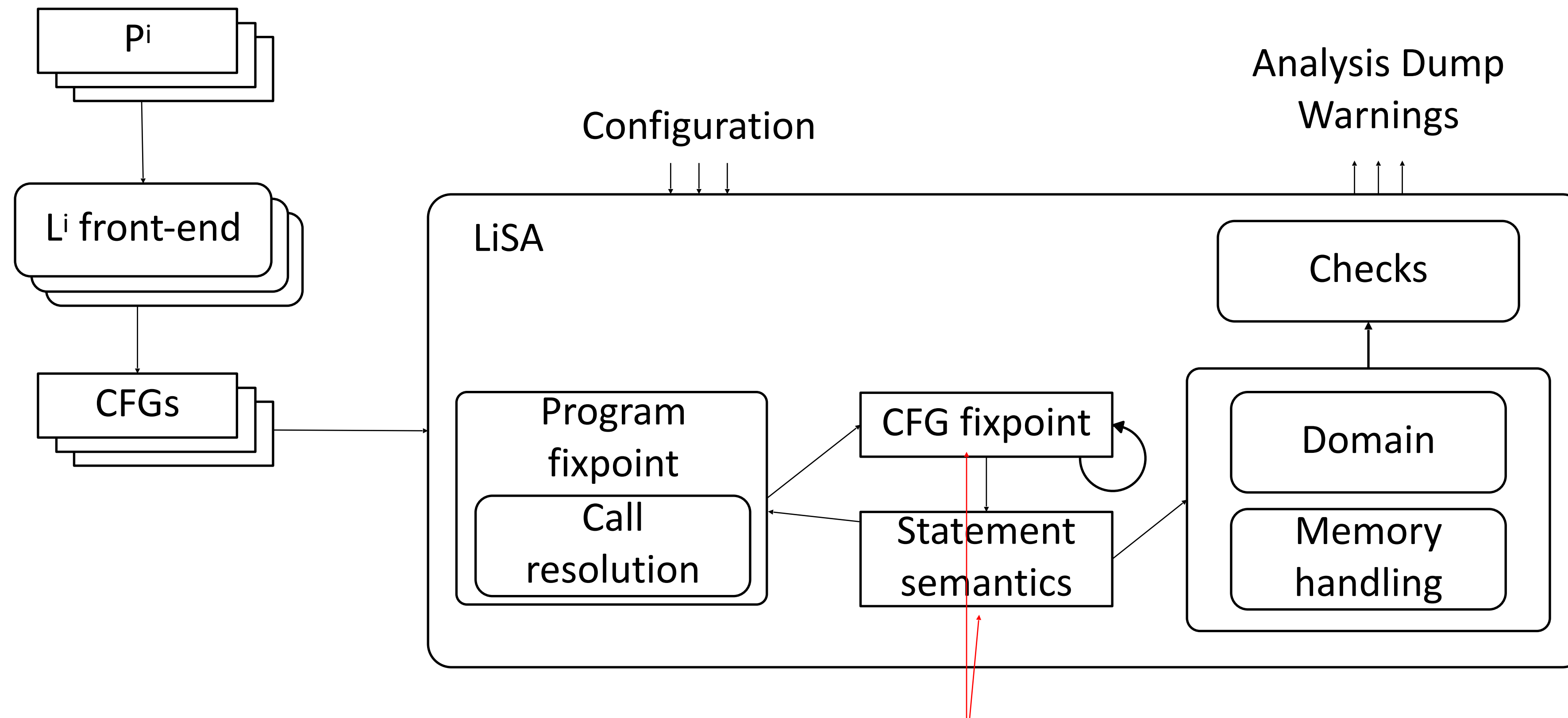


LiSA overview



How do these work?

LiSA overview



How do these work?

CFG's fixpoint

```
result  $\leftarrow \{\}$   
ws  $\leftarrow \{st_0\}$   
while ws  $\neq \emptyset$  do  
  st  $\leftarrow pop(ws)$   
  inst  $\leftarrow \sqcup \{out_m : m \in preds(st)\}$   
  outst  $\leftarrow semantics_{st}(in_{st})$   
  if not outst  $\leq result[st]$  then  
    if widening threshold reached then  
      result[st]  $\leftarrow result[st] \nabla out_{st}$   
    else  
      result[st]  $\leftarrow result[st] \sqcup out_{st}$   
    end if  
    ws  $\leftarrow ws \cup succ(st)$   
  end if  
end while
```

CFG's fixpoint

```
result  $\leftarrow \{\}$   
ws  $\leftarrow \{st_0\}$   
while  $ws \neq \emptyset$  do  
   $st \leftarrow pop(ws)$   
   $in_{st} \leftarrow \sqcup \{out_m : m \in preds(st)\}$   
   $out_{st} \leftarrow semantics_{st}(in_{st})$   
  if not  $out_{st} \leq result[st]$  then  
    if widening threshold reached then  
       $result[st] \leftarrow result[st] \nabla out_{st}$   
    else  
       $result[st] \leftarrow result[st] \sqcup out_{st}$   
    end if  
     $ws \leftarrow ws \cup succ(st)$   
  end if  
end while
```

Lub: over-approximate entry state

CFG's fixpoint

```
result  $\leftarrow \{\}$   
ws  $\leftarrow \{st_0\}$   
while  $ws \neq \emptyset$  do  
  st  $\leftarrow pop(ws)$   
   $in_{st} \leftarrow \sqcup \{out_m : m \in preds(st)\}$   
   $out_{st} \leftarrow semantics_{st}(in_{st})$   
  if not  $out_{st} \leq result[st]$  then  
    if widening threshold reached then  
       $result[st] \leftarrow result[st] \nabla out_{st}$   
    else  
       $result[st] \leftarrow result[st] \sqcup out_{st}$   
    end if  
  end if  
   $ws \leftarrow ws \cup succ(st)$   
end if  
end while
```

Lub: over-approximate entry state

Partial order: keep “highest” result

CFG's fixpoint

```
result  $\leftarrow \{\}$ 
ws  $\leftarrow \{st_0\}$ 
while ws  $\neq \emptyset$  do
  st  $\leftarrow pop(ws)$ 
  inst  $\leftarrow \sqcup \{out_m : m \in preds(st)\}$ 
  outst  $\leftarrow semantics_{st}(in_{st})$ 
  if not outst  $\leq result[st]$  then
    if widening threshold reached then
      result[st]  $\leftarrow result[st] \nabla out_{st}$ 
    else
      result[st]  $\leftarrow result[st] \sqcup out_{st}$ 
    end if
  end if
  ws  $\leftarrow ws \cup succ(st)$ 
end if
end while
```

Lub: over-approximate entry state

Partial order: keep “highest” result

Lub: move “upwards”

CFG's fixpoint

```
result  $\leftarrow \{\}$   
ws  $\leftarrow \{st_0\}$   
while ws  $\neq \emptyset$  do  
  st  $\leftarrow pop(ws)$   
  inst  $\leftarrow \sqcup \{out_m : m \in preds(st)\}$   
  outst  $\leftarrow semantics_{st}(in_{st})$   
  if not outst  $\leq result[st]$  then  
    if widening threshold reached then  
      result[st]  $\leftarrow result[st] \nabla out_{st}$   
    else  
      result[st]  $\leftarrow result[st] \sqcup out_{st}$   
    end if  
    ws  $\leftarrow ws \cup succ(st)$   
  end if  
end while
```

Lub: over-approximate entry state

Partial order: keep “highest” result

Widening: force convergence

Lub: move “upwards”

CFG's fixpoint

```
result  $\leftarrow \{\}$ 
ws  $\leftarrow \{st_0\}$ 
while ws  $\neq \emptyset$  do
  st  $\leftarrow pop(ws)$ 
  inst  $\leftarrow \sqcup \{out_m : m \in preds(st)\}$ 
  outst  $\leftarrow semantics_{st}(in_{st})$ 
  if not outst  $\leq result[st]$  then
    if widening threshold reached then
      result[st]  $\leftarrow result[st] \nabla out_{st}$ 
    else
      result[st]  $\leftarrow result[st] \sqcup out_{st}$ 
    end if
  end if
  ws  $\leftarrow ws \cup succ(st)$ 
end if
end while
```

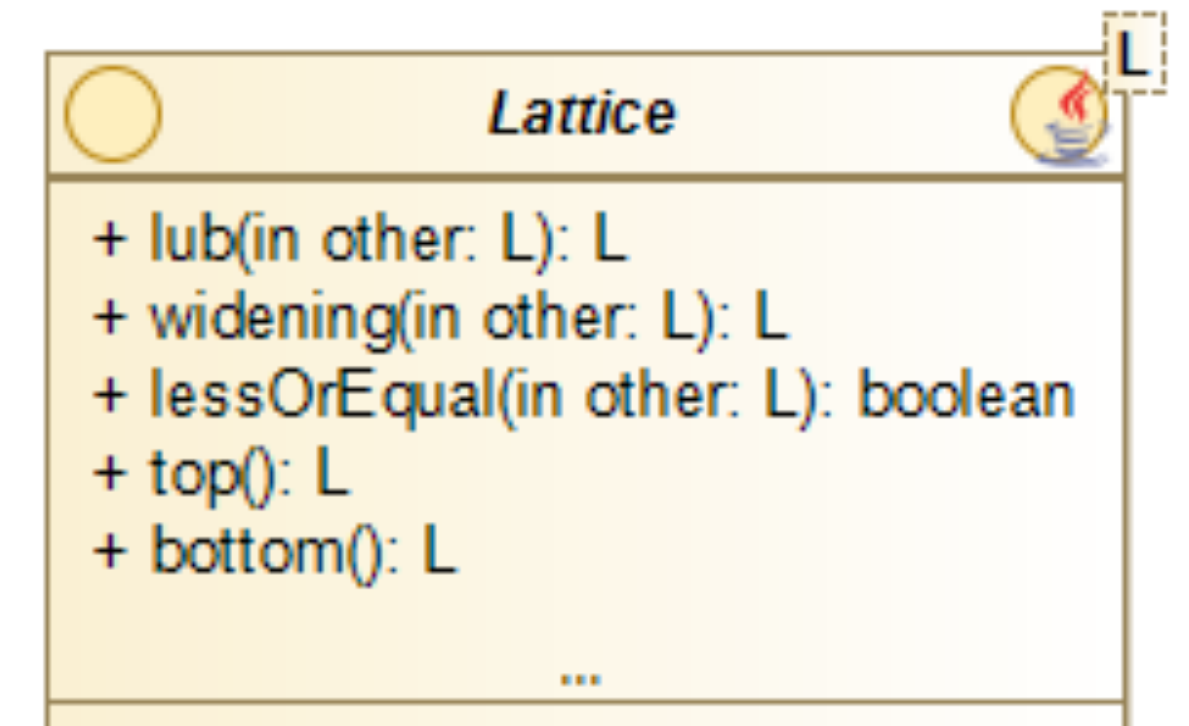
Lub: over-approximate entry state

Partial order: keep “highest” result

Widening: force convergence

Lub: move “upwards”

States must be lattices!

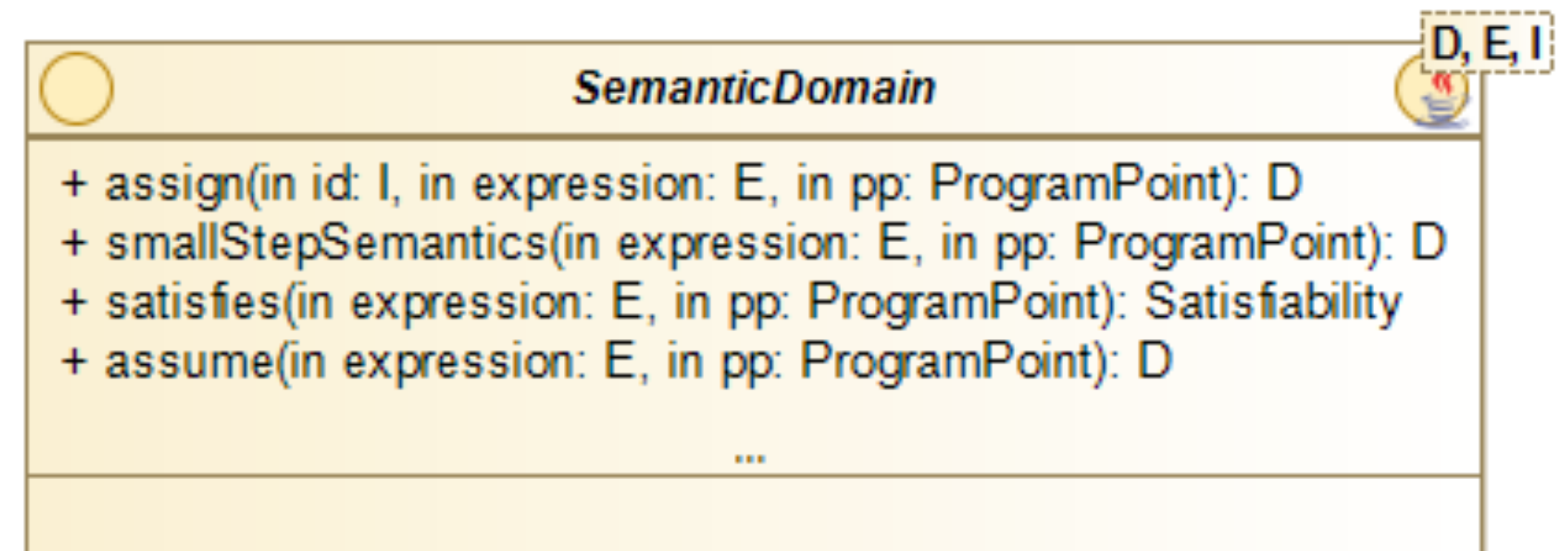


Statement's semantics

3 possible operations:

- Assignment
- Expression evaluation
- Condition traversing

States must be semantic domains!



Statement's semantics

Addition.semantics():

```
return domain.smallStepSemantics(new ArithmSum(left, right))
```

Assignment.semantics():

```
return domain.assign(left, right)
```

Conditional.semantics():

```
tbranch = domain.assume(cond).smallStepSemantics(left)
```

```
fbranch = domain.assume(cond.negate()).smallStepSemantics(right)
```

```
if domain.satisfies(cond) == SATISFIED:
```

```
    return tbranch
```

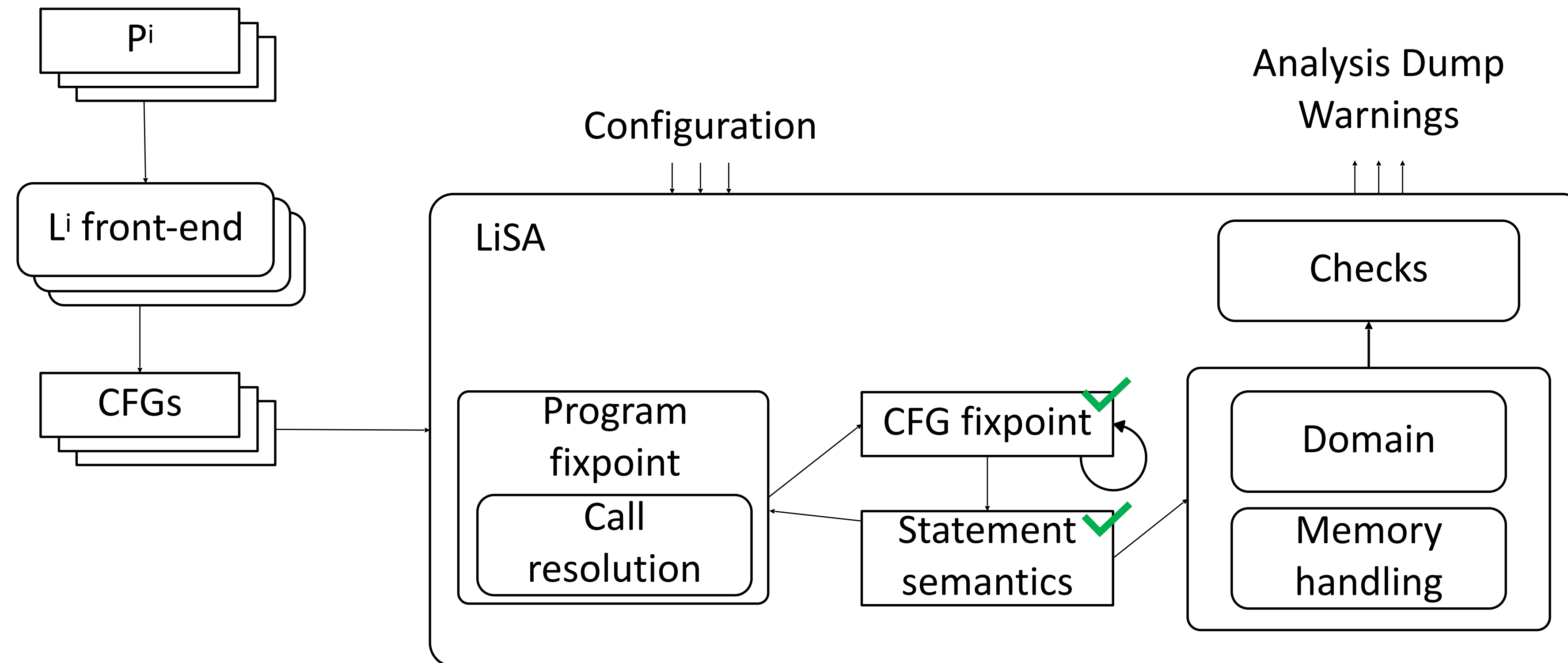
```
else if domain.satisfies(cond) == NOT_SATISFIED:
```

```
    return fbranch
```

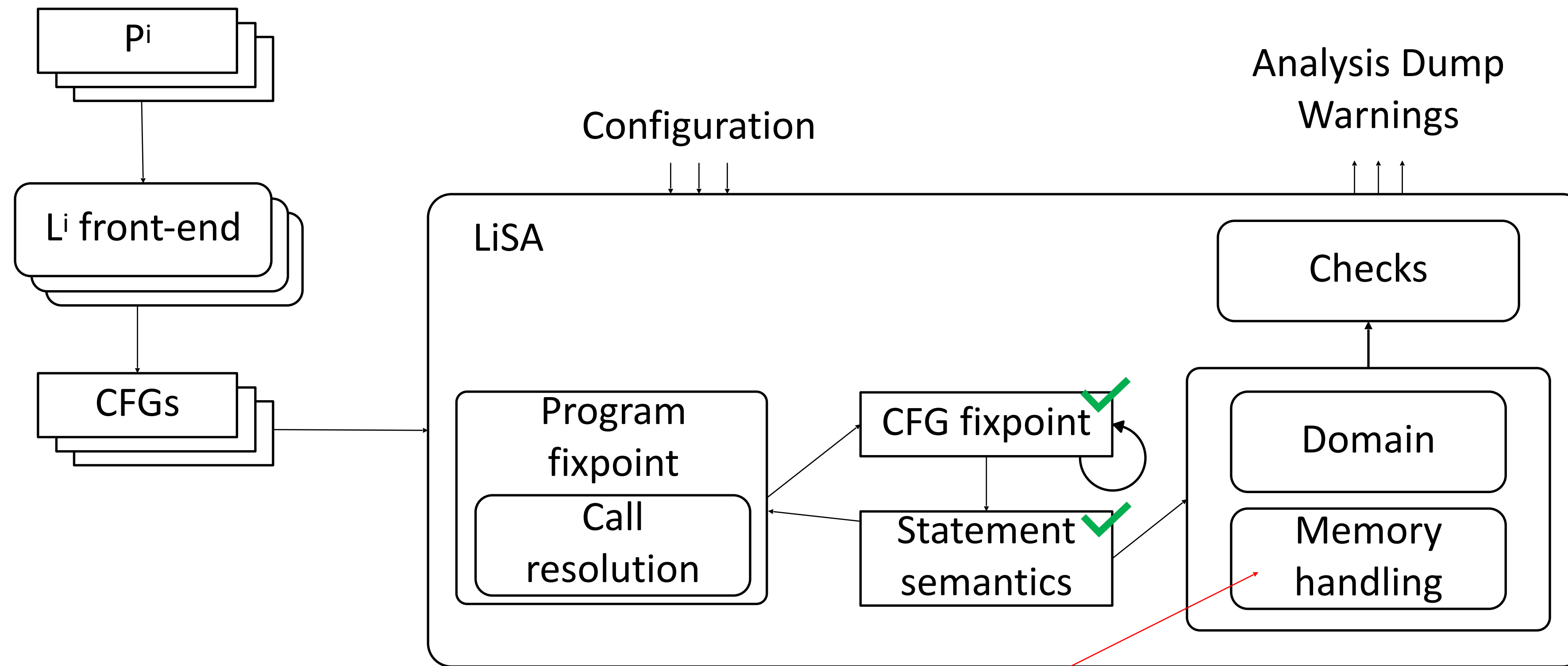
```
else:
```

```
    return tbranch.lub(fbranch)
```


LiSA overview



LiSA overview

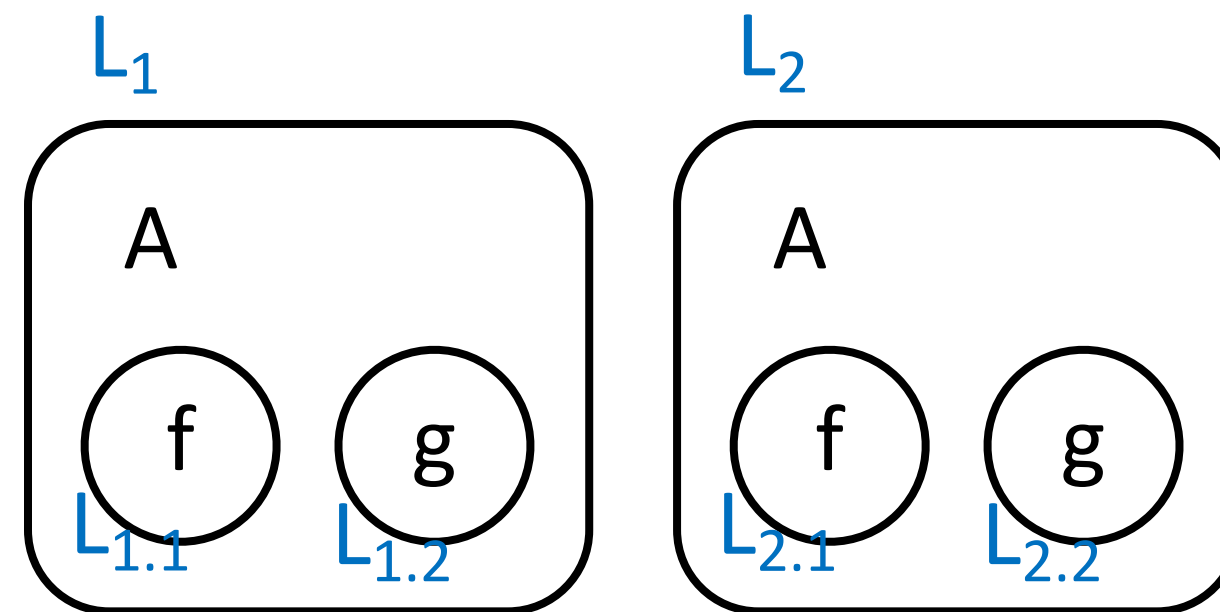


How do this work?

Handling dynamic memory

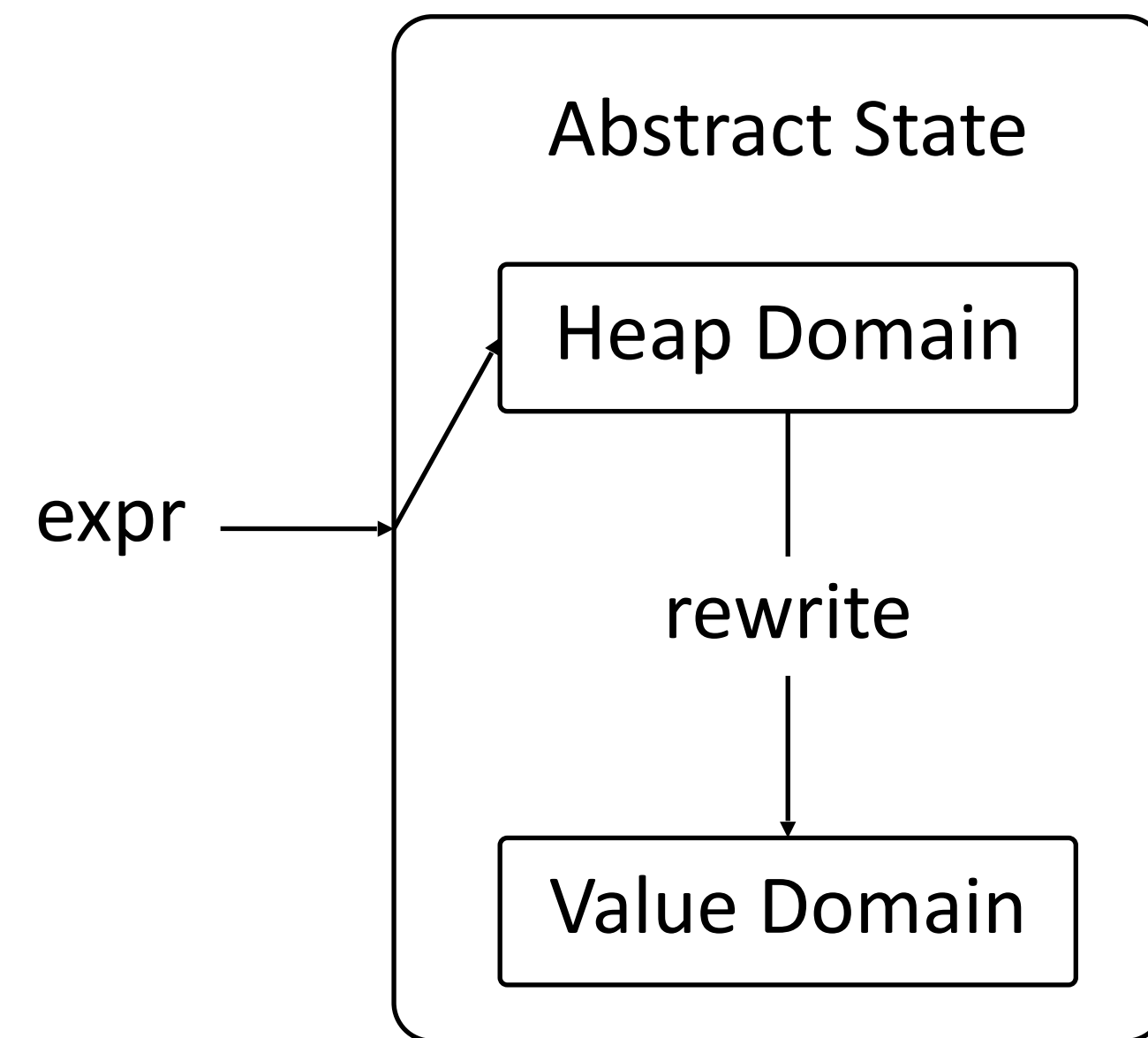
- We want domains to be independent on how we handle memory
- Intuition: we can consider each memory location as a “fake” variable
 - Handling dynamic memory access means understanding to which location an expression refers to

```
class A {  
    int f, g;  
}  
x = new A();  
y = new A();
```

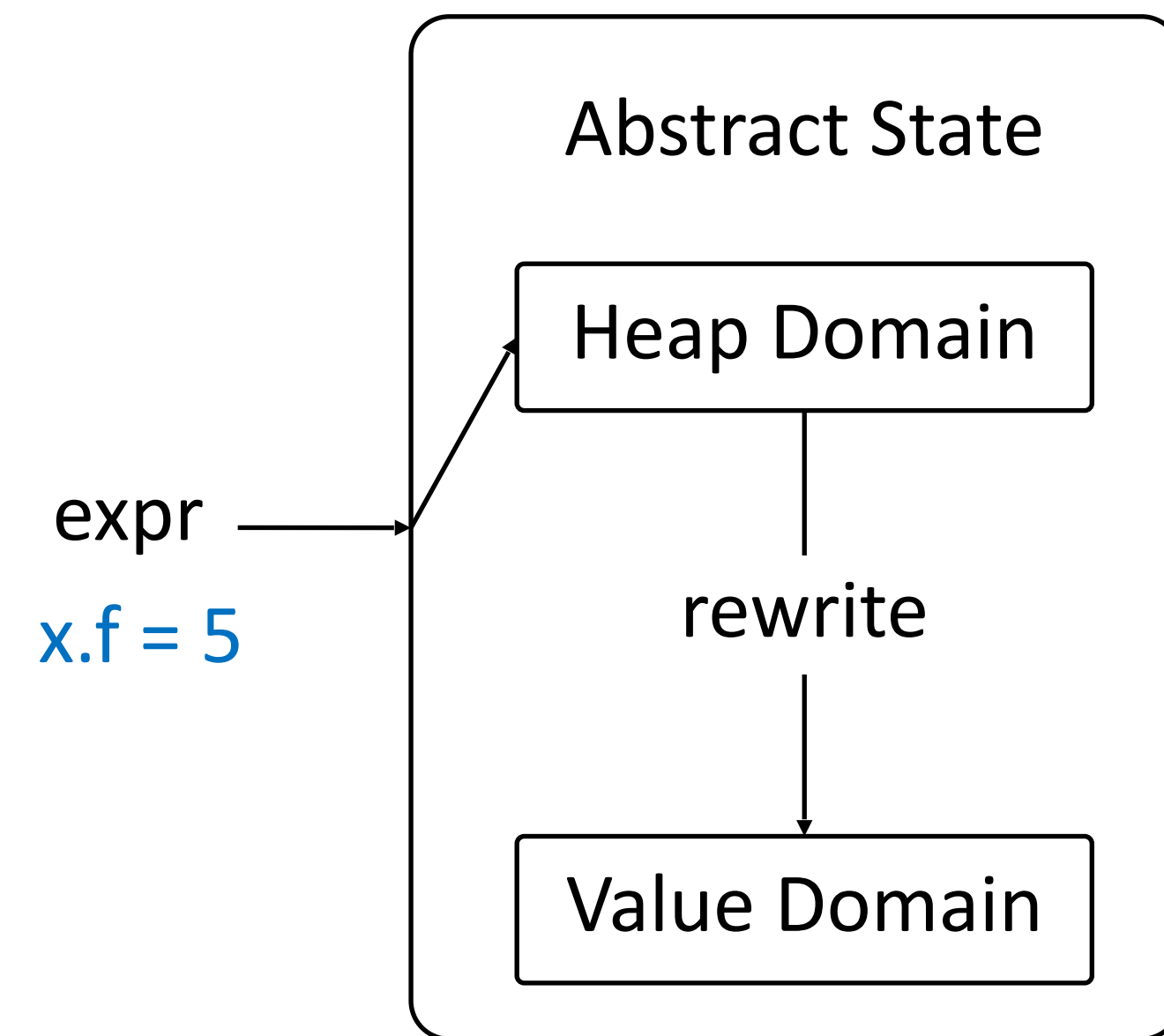


Expression	Concrete location
x	L_1
$x.f$	$L_{1.1}$
$x.g$	$L_{1.2}$
y	L_2
$y.f$	$L_{2.1}$
$y.g$	$L_{2.2}$

The Abstract State

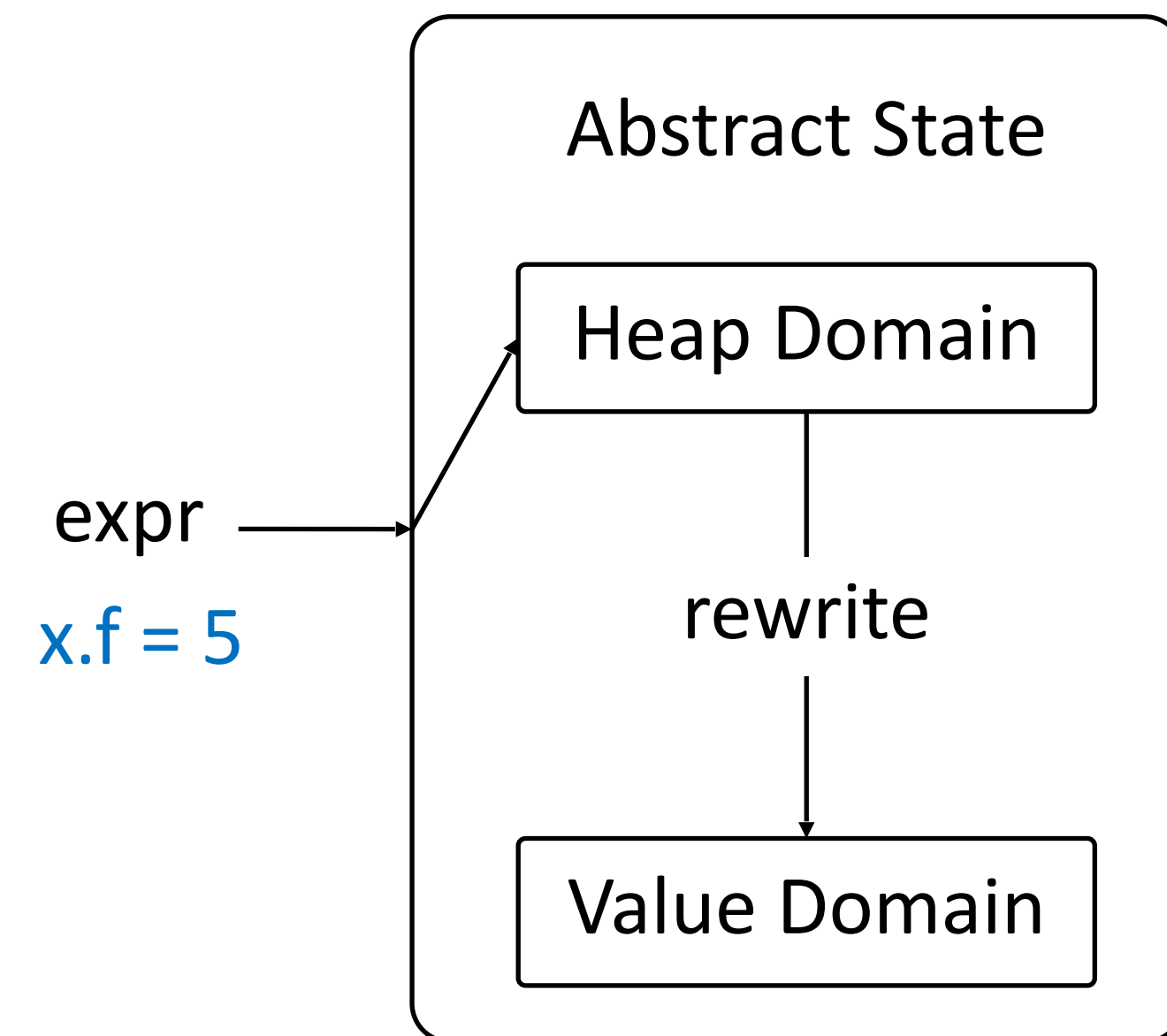


The Abstract State

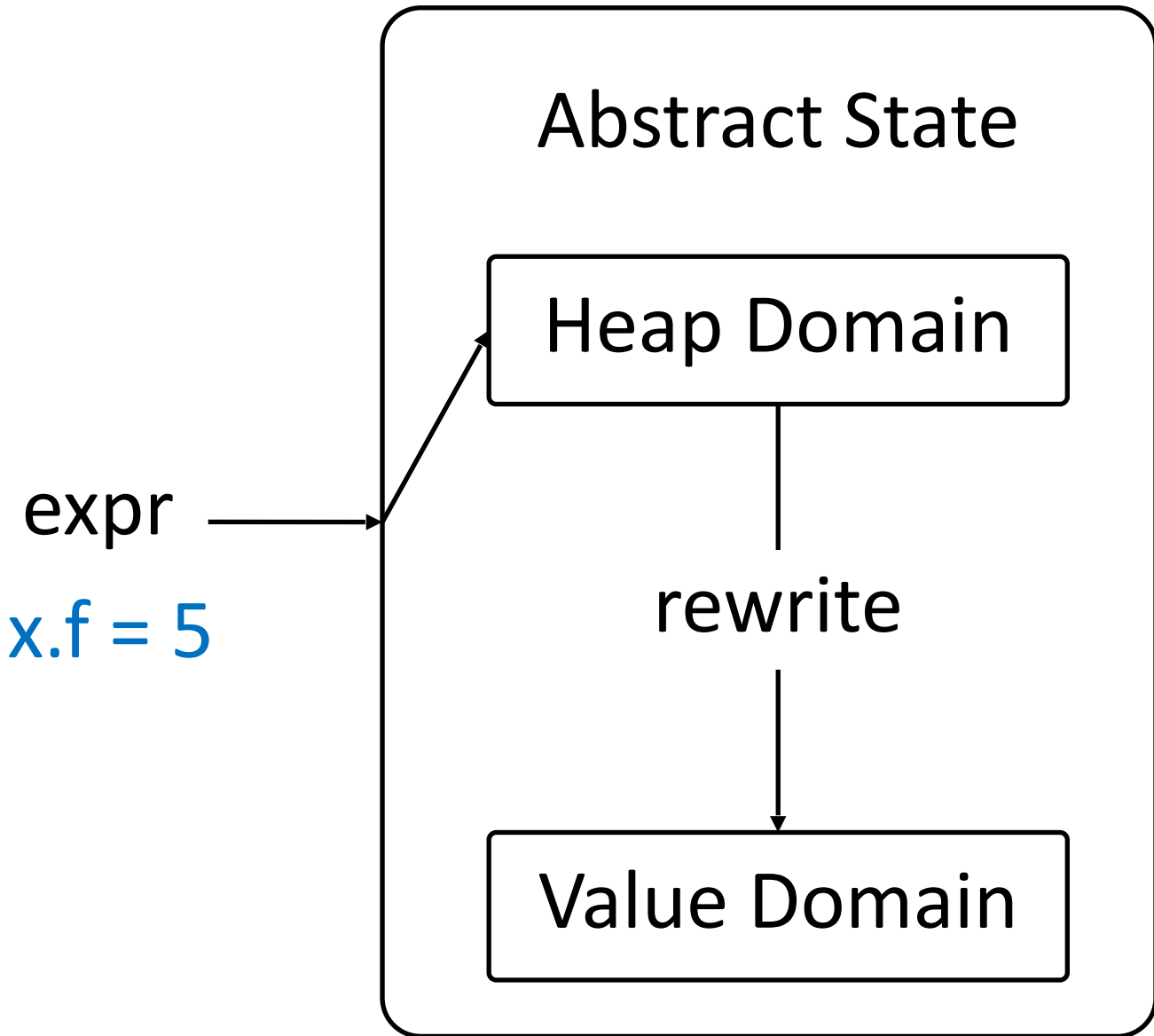


The Abstract State

How much precision do we want?



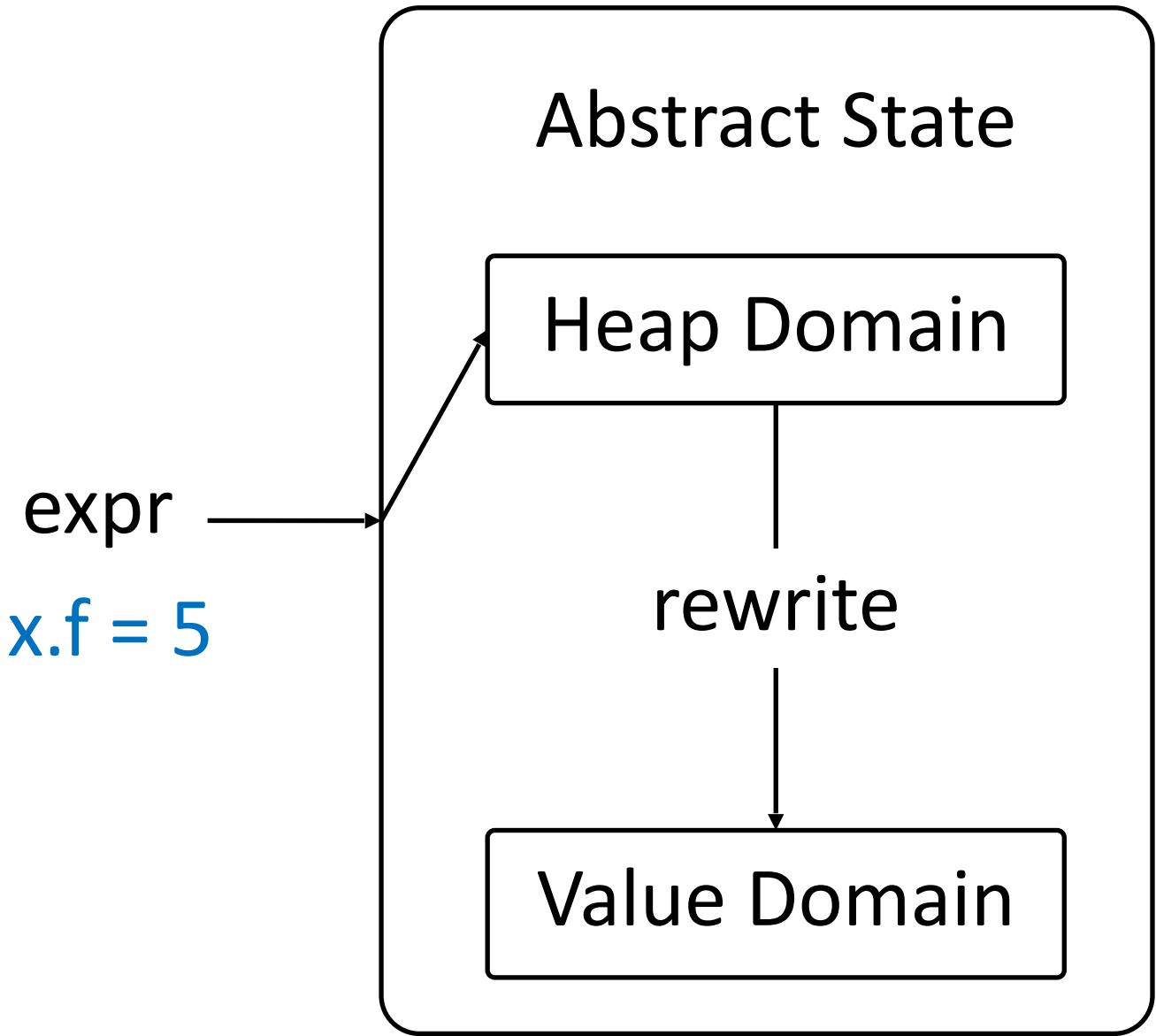
The Abstract State



How much precision do we want?

Expression	Concrete location	Low precision	Medium precision	High precision
x	L ₁	L	L ₁	L ₁
x.f	L _{1.1}	L	L ₁	L _{1.1}
x.g	L _{1.2}	L	L ₁	L _{1.2}
y	L ₂	L	L ₂	L ₂
y.f	L _{2.1}	L	L ₂	L _{2.1}
y.g	L _{2.2}	L	L ₂	L _{2.2}

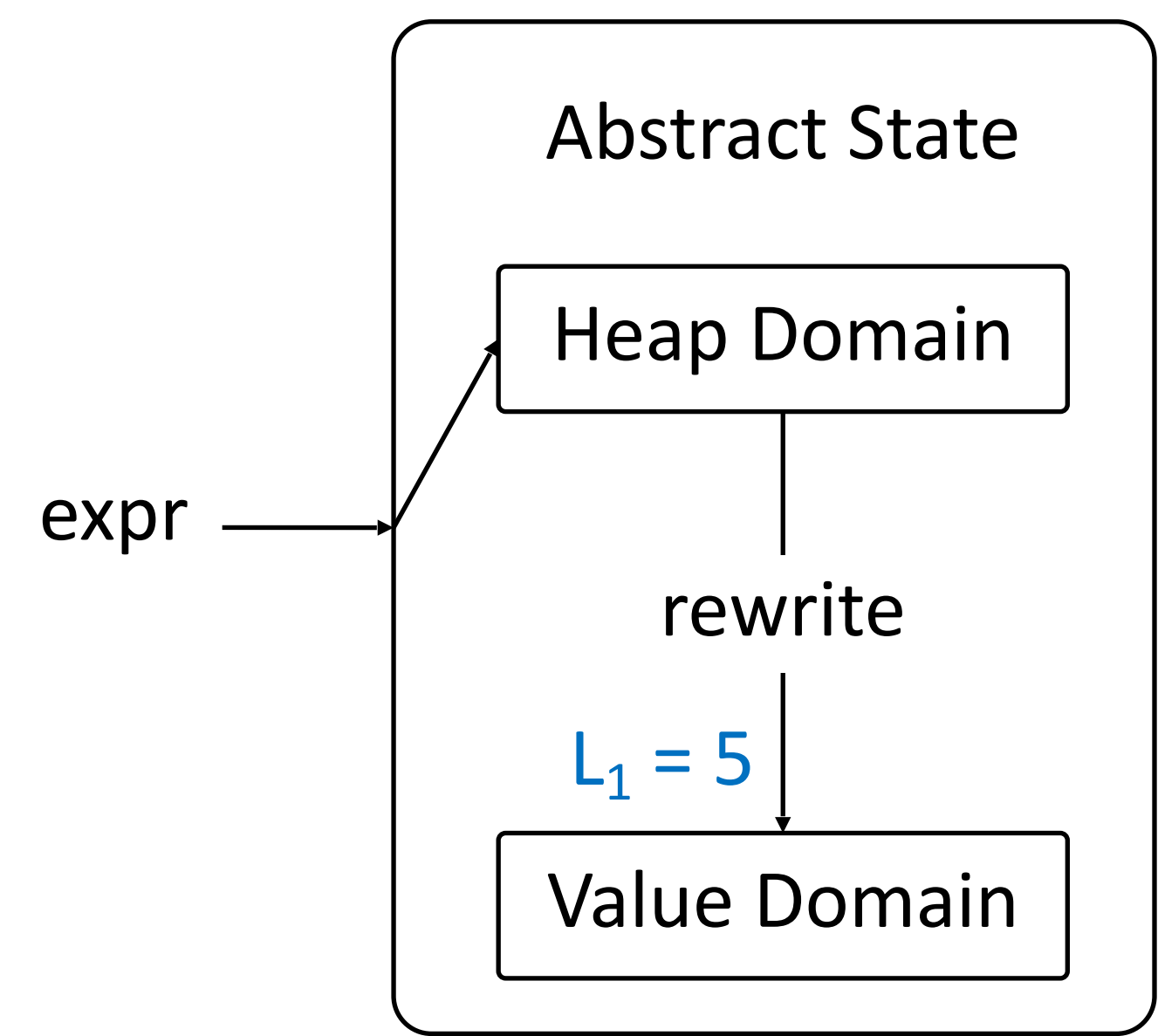
The Abstract State



How much precision do we want?

Expression	Concrete location	Low precision	Medium precision	High precision
x	L ₁	L	L ₁	L ₁
x.f	L _{1.1}	L	L ₁	L _{1.1}
x.g	L _{1.2}	L	L ₁	L _{1.2}
y	L ₂	L	L ₂	L ₂
y.f	L _{2.1}	L	L ₂	L _{2.1}
y.g	L _{2.2}	L	L ₂	L _{2.2}

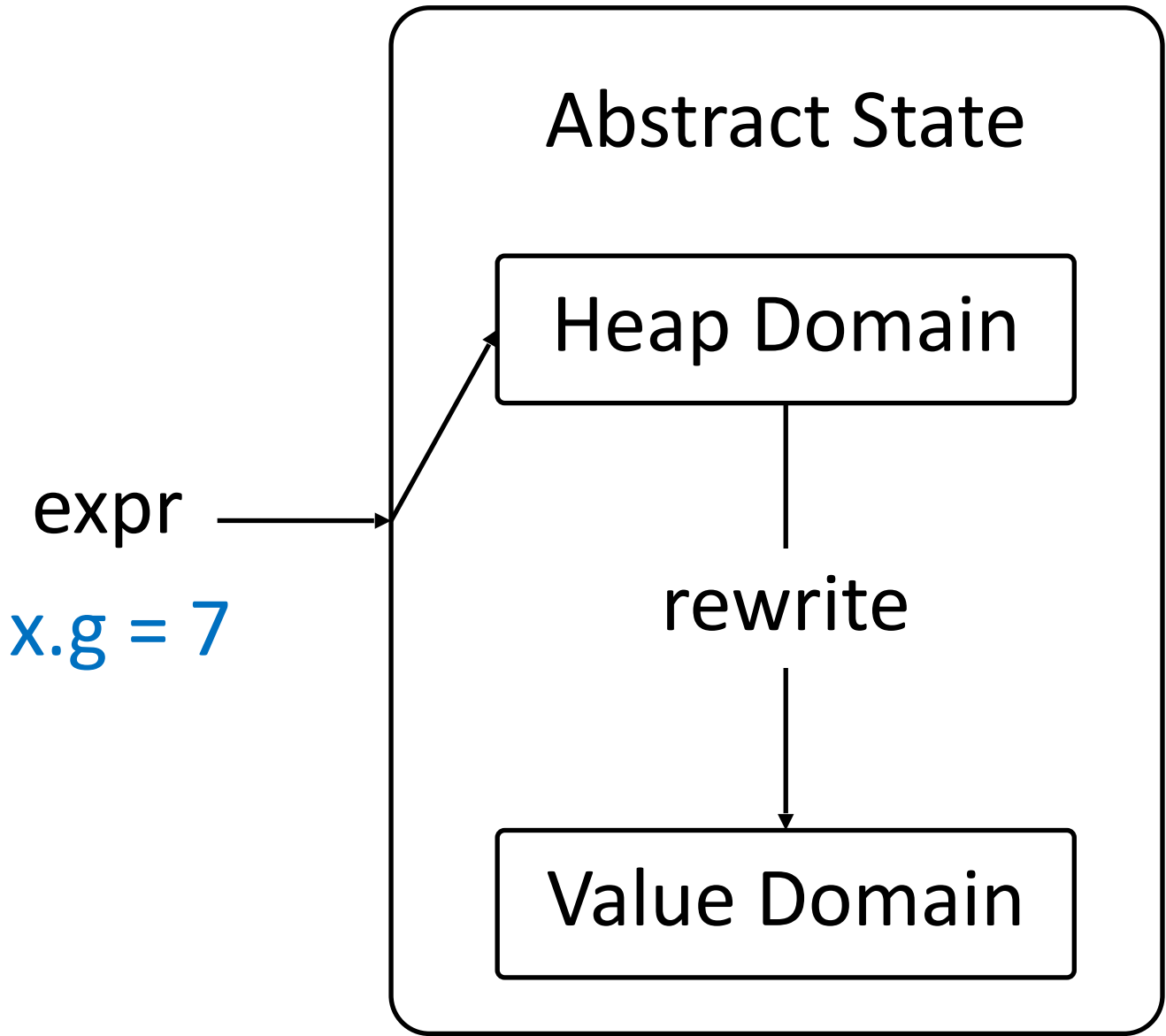
The Abstract State



How much precision do we want?

Expression	Concrete location	Low precision	Medium precision	High precision
x	L_1	L	L_1	L_1
x.f	$L_{1.1}$	L	L_1	$L_{1.1}$
x.g	$L_{1.2}$	L	L_1	$L_{1.2}$
y	L_2	L	L_2	L_2
y.f	$L_{2.1}$	L	L_2	$L_{2.1}$
y.g	$L_{2.2}$	L	L_2	$L_{2.2}$

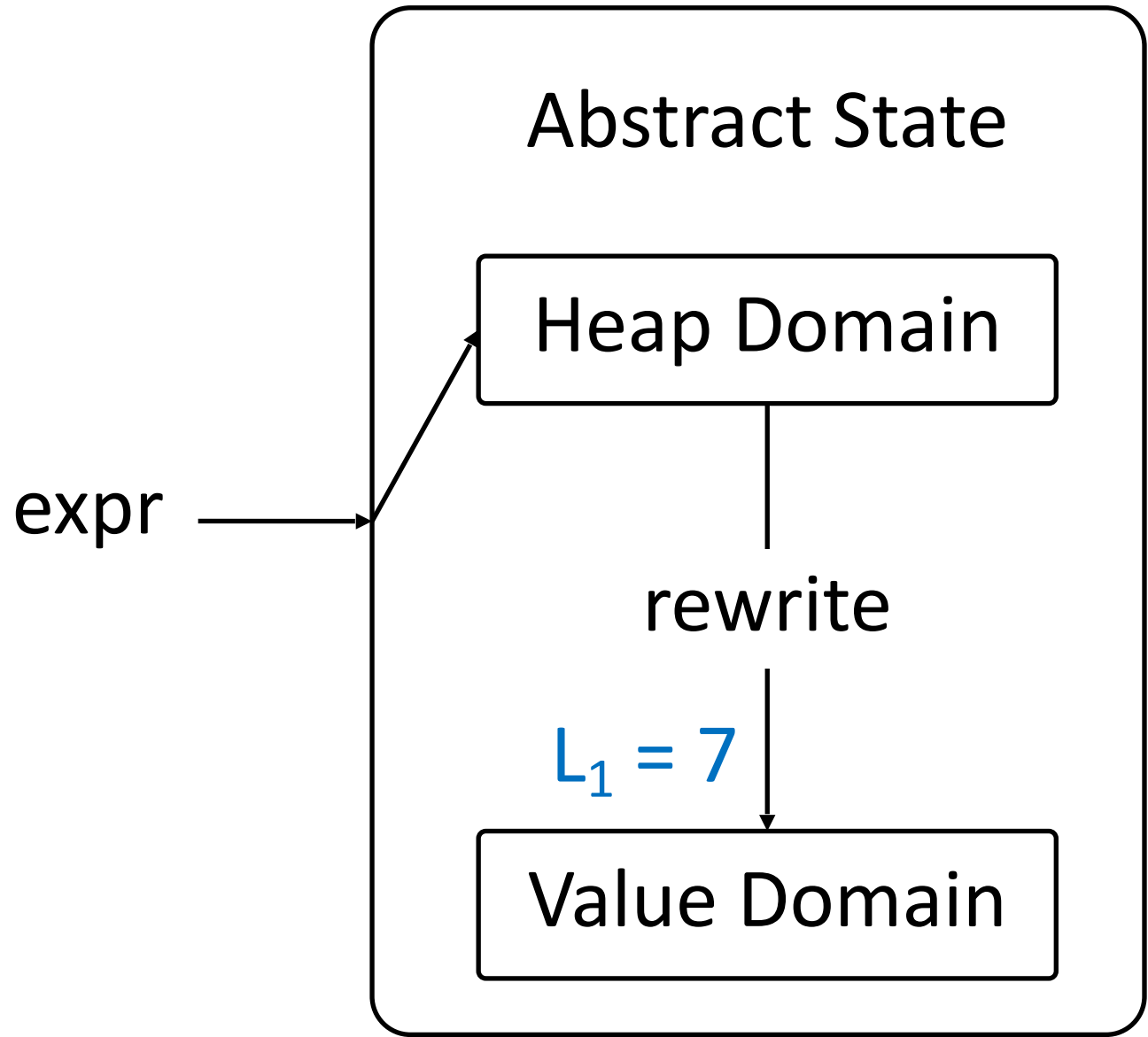
The Abstract State



How much precision do we want?

Expression	Concrete location	Low precision	Medium precision	High precision
x	L ₁	L	L ₁	L ₁
x.f	L _{1.1}	L	L ₁	L _{1.1}
x.g	L _{1.2}	L	L ₁	L _{1.2}
y	L ₂	L	L ₂	L ₂
y.f	L _{2.1}	L	L ₂	L _{2.1}
y.g	L _{2.2}	L	L ₂	L _{2.2}

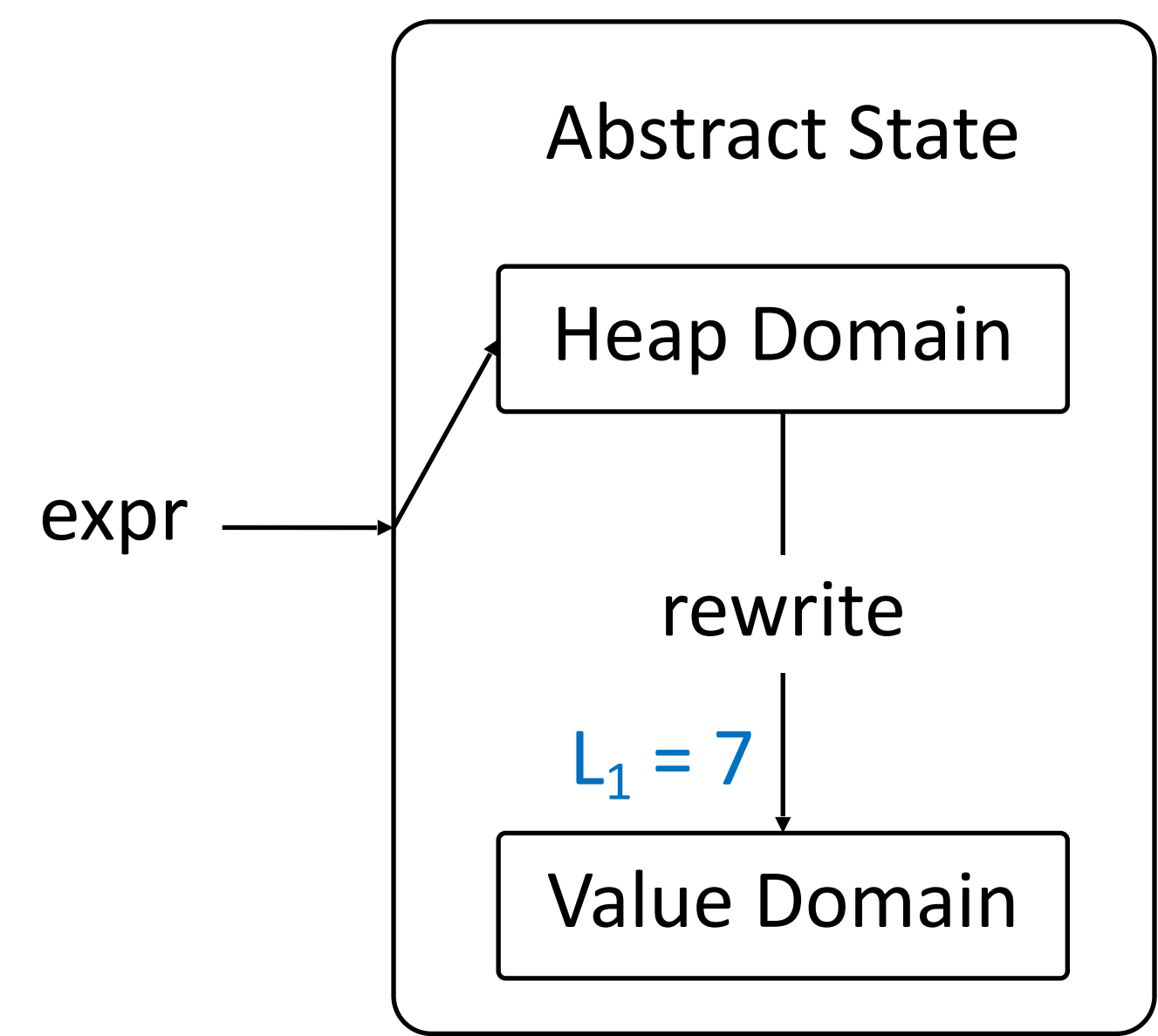
The Abstract State



How much precision do we want?

Expression	Concrete location	Low precision	Medium precision	High precision
x	L_1	L	L_1	L_1
x.f	$L_{1.1}$	L	L_1	$L_{1.1}$
x.g	$L_{1.2}$	L	L_1	$L_{1.2}$
y	L_2	L	L_2	L_2
y.f	$L_{2.1}$	L	L_2	$L_{2.1}$
y.g	$L_{2.2}$	L	L_2	$L_{2.2}$

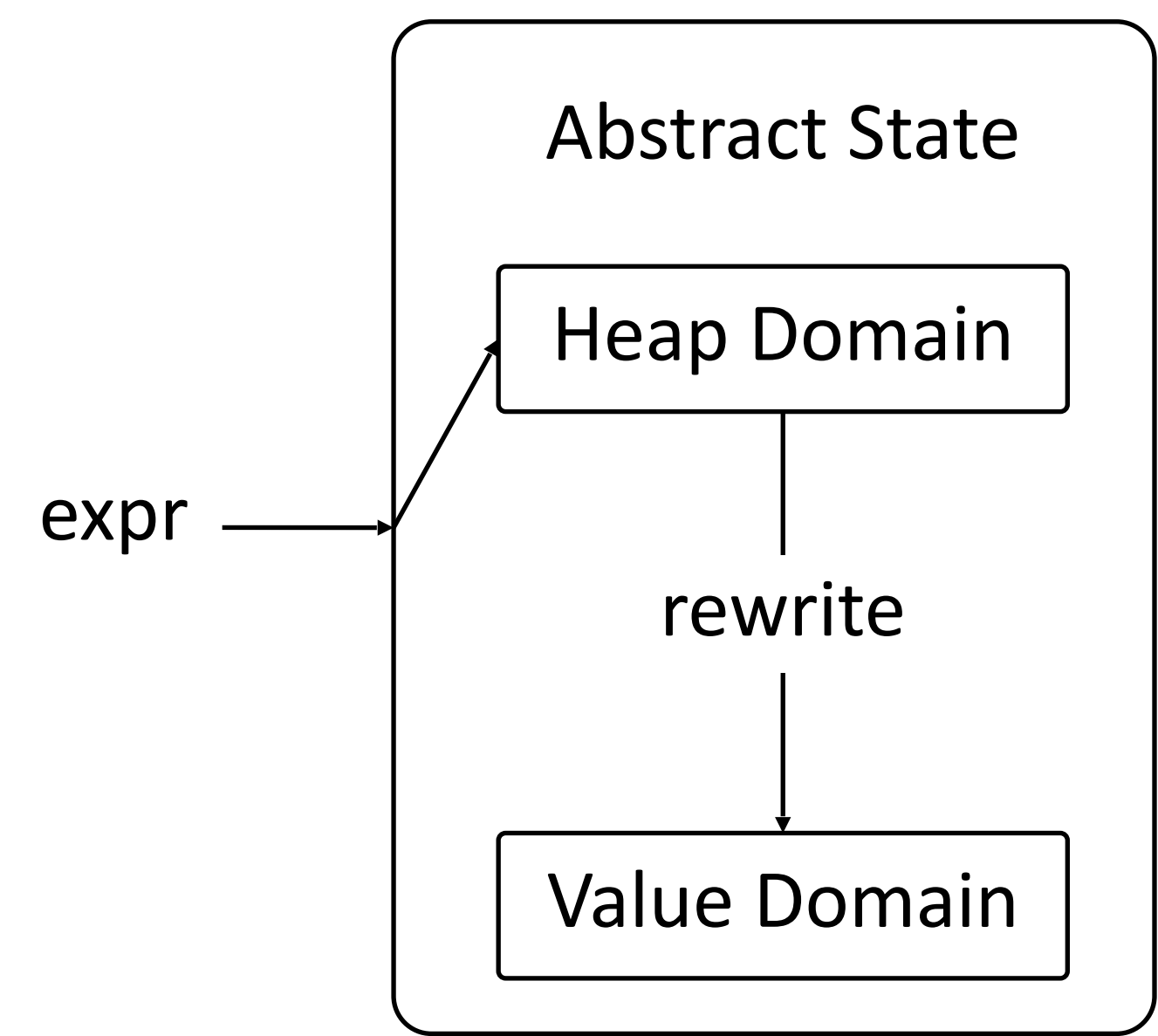
The Abstract State



Can we do better?

Expression	Concrete location	Low precision	Medium precision	High precision
x	L ₁	L	L ₁	L ₁
x.f	L _{1.1}	L	L ₁	L _{1.1}
x.g	L _{1.2}	L	L ₁	L _{1.2}
y	L ₂	L	L ₂	L ₂
y.f	L _{2.1}	L	L ₂	L _{2.1}
y.g	L _{2.2}	L	L ₂	L _{2.2}

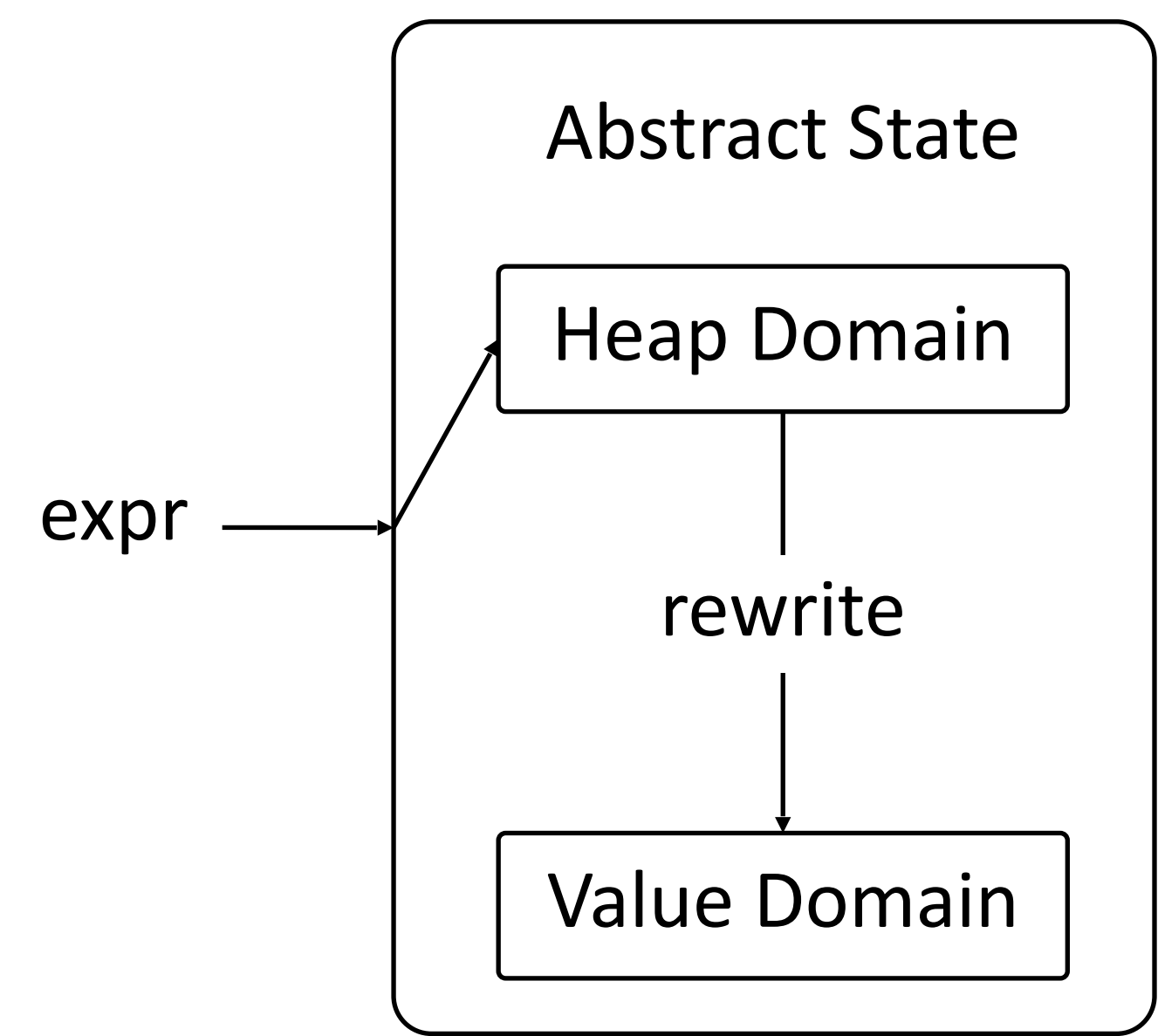
The Abstract State



Can we do better?

Expression	Concrete location	Low precision	Medium precision	High precision
x	L ₁	L	L ₁	L ₁
x.f	L _{1.1}	L	L ₁	L _{1.1}
x.g	L _{1.2}	L	L ₁	L _{1.2}
y	L ₂	L	L ₂	L ₂
y.f	L _{2.1}	L	L ₂	L _{2.1}
y.g	L _{2.2}	L	L ₂	L _{2.2}

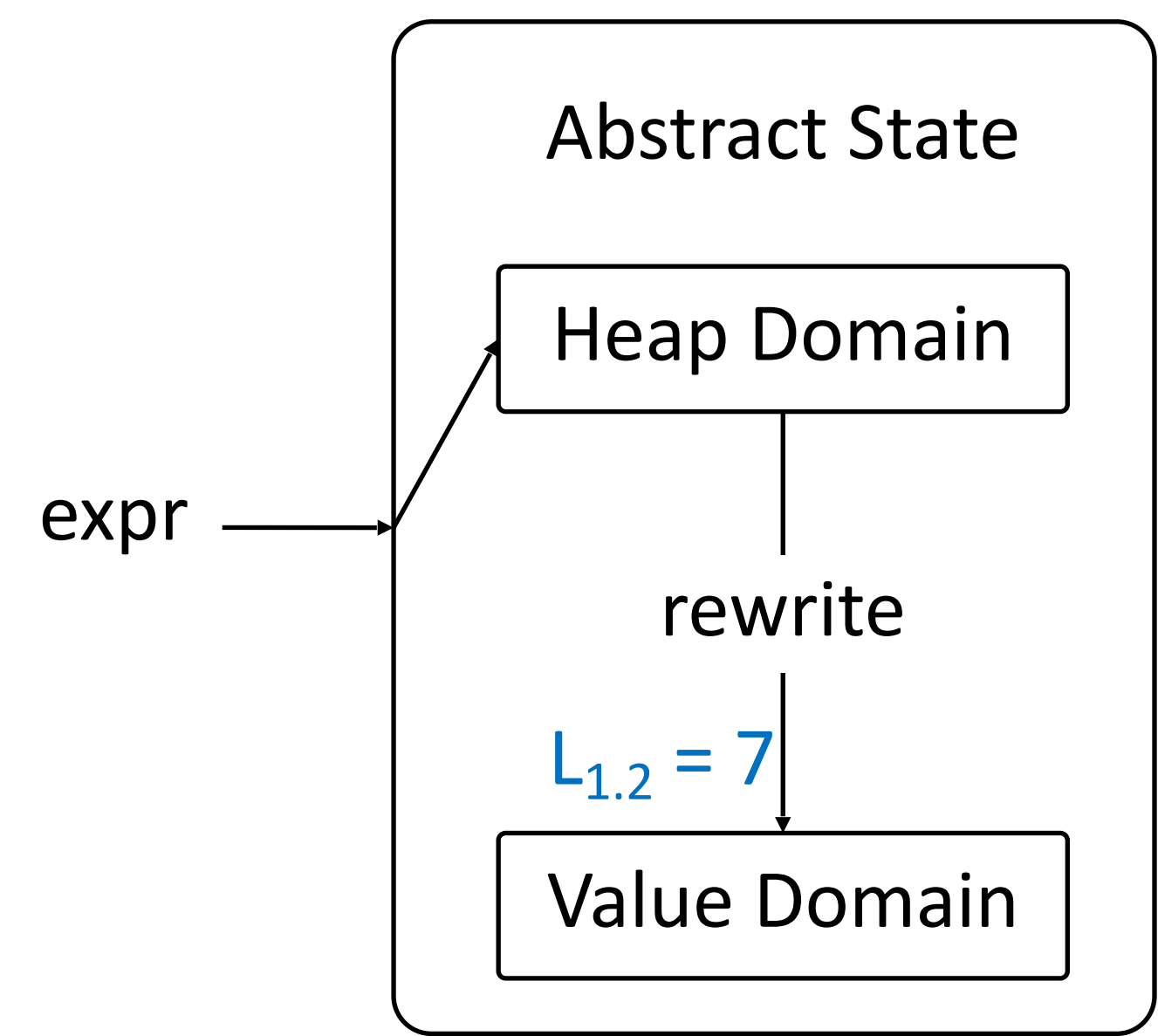
The Abstract State



Can we do better?

Expression	Concrete location	Low precision	Medium precision	High precision
x	L ₁	L	L ₁	L ₁
x.f	L _{1.1}	L	L ₁	L _{1.1}
x.g	L _{1.2}	L	L ₁	L _{1.2}
y	L ₂	L	L ₂	L ₂
y.f	L _{2.1}	L	L ₂	L _{2.1}
y.g	L _{2.2}	L	L ₂	L _{2.2}

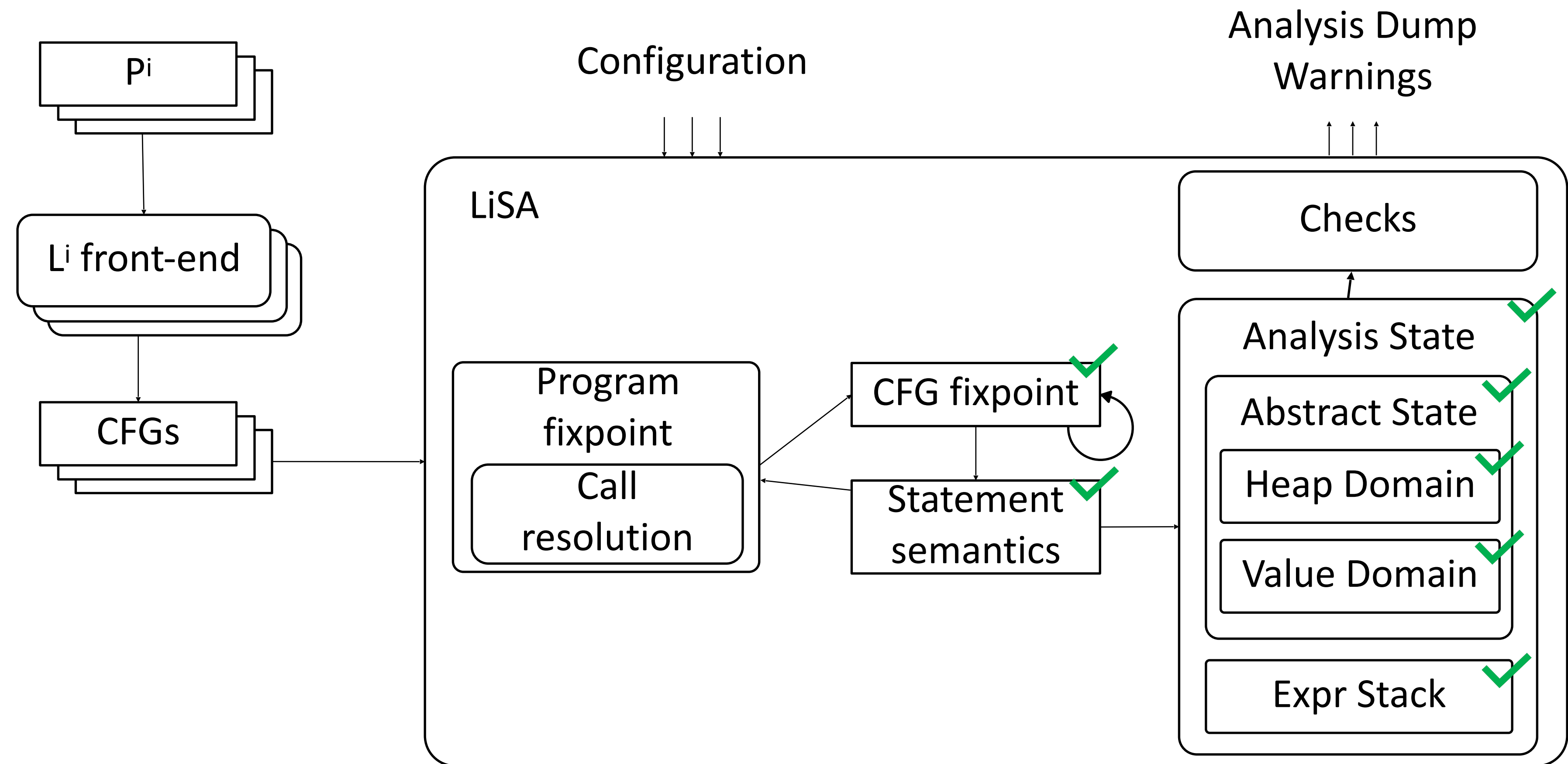
The Abstract State



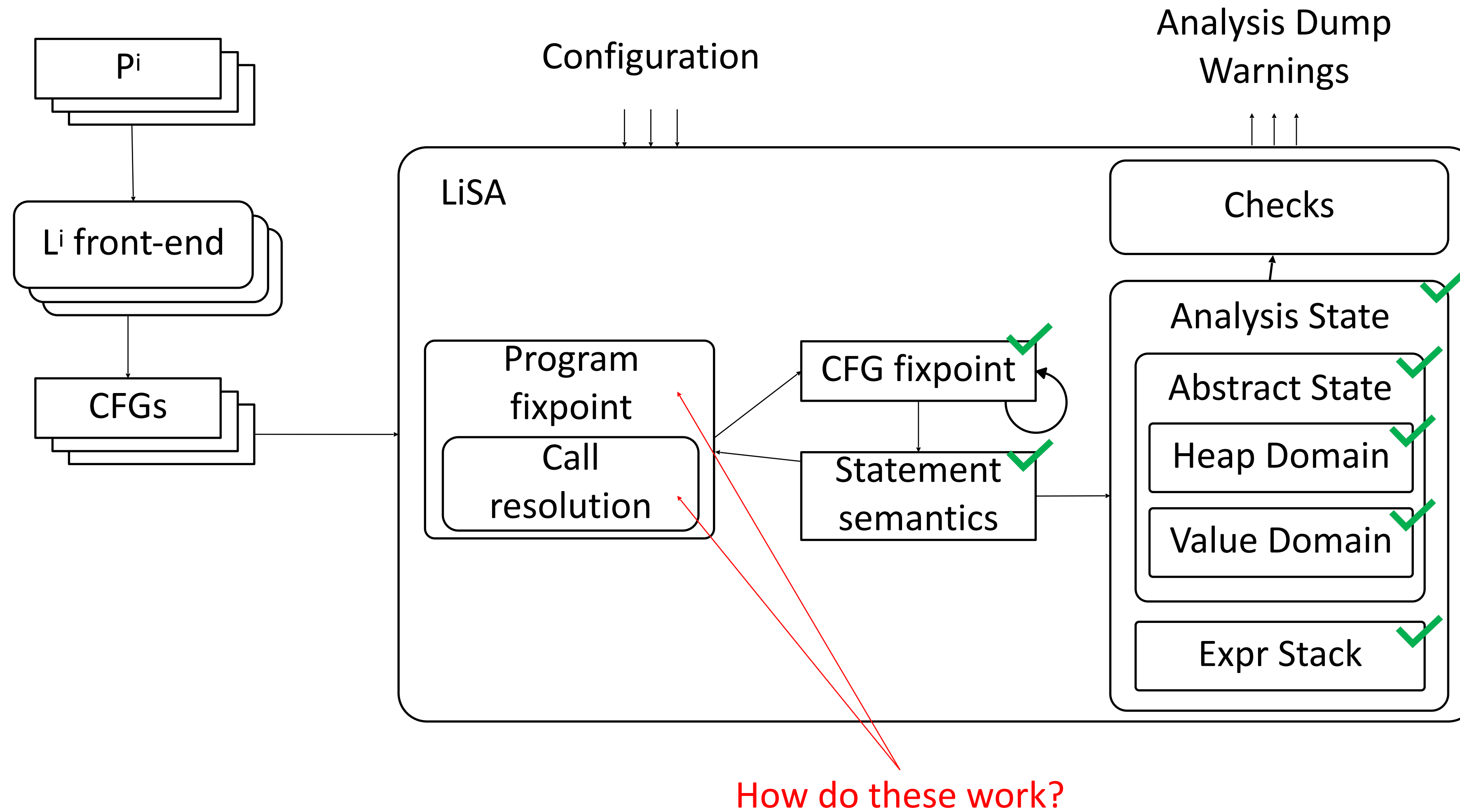
Can we do better?

Expression	Concrete location	Low precision	Medium precision	High precision
x	L_1	L	L_1	L_1
x.f	$L_{1.1}$	L	L_1	$L_{1.1}$
x.g	$L_{1.2}$	L	L_1	$L_{1.2}$
y	L_2	L	L_2	L_2
y.f	$L_{2.1}$	L	L_2	$L_{2.1}$
y.g	$L_{2.2}$	L	L_2	$L_{2.2}$

LiSA overview



LiSA overview



The Interprocedural Analysis

Interprocedural
Analysis

The Interprocedural Analysis



Interprocedural analysis can:

- analyze CFGs in isolation
- start from “main” and follow calls
- ...

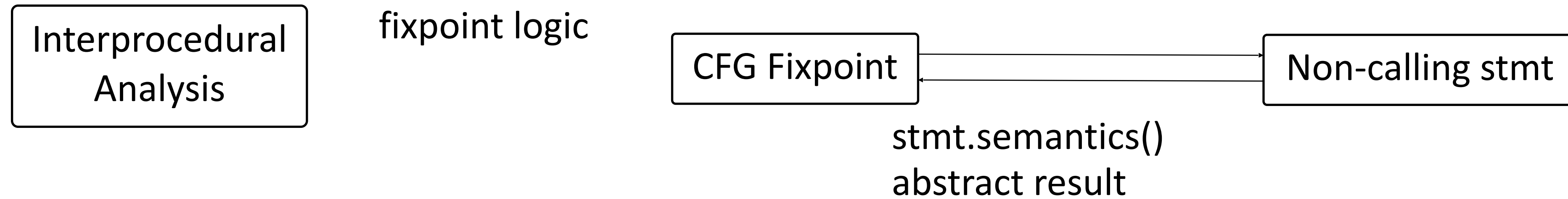
The Interprocedural Analysis



Interprocedural analysis can:

- analyze CFGs in isolation
- start from “main” and follow calls
- ...

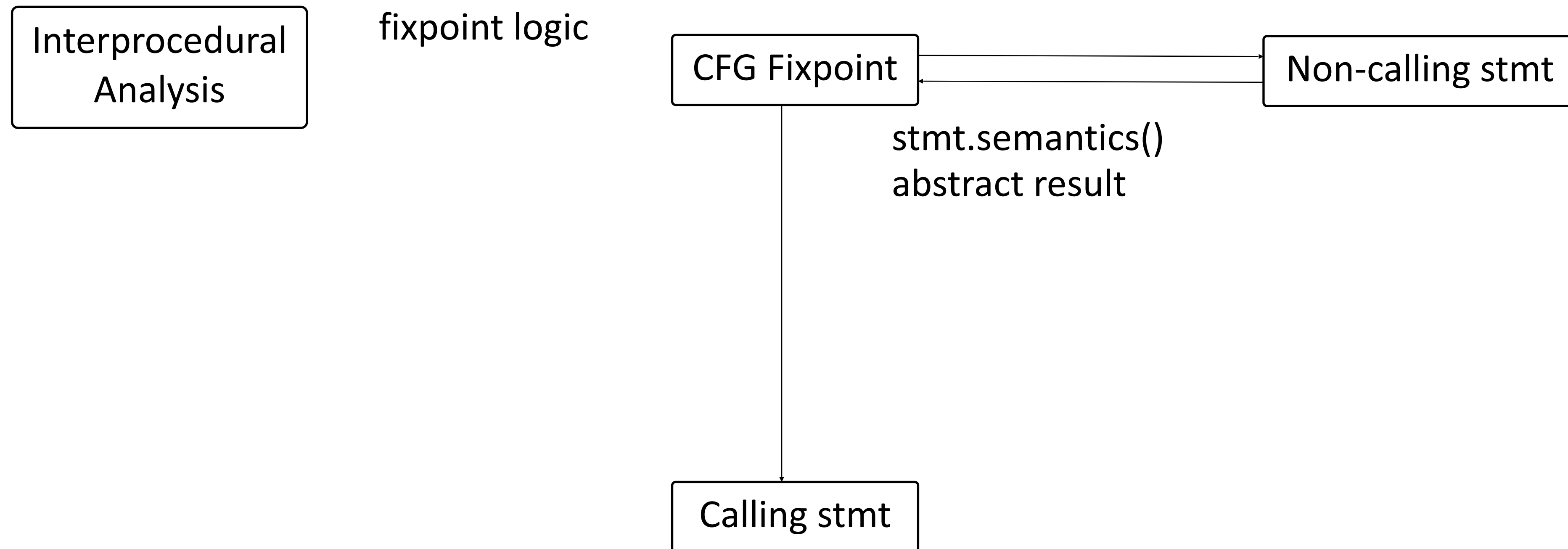
The Interprocedural Analysis



Interprocedural analysis can:

- analyze CFGs in isolation
- start from “main” and follow calls
- ...

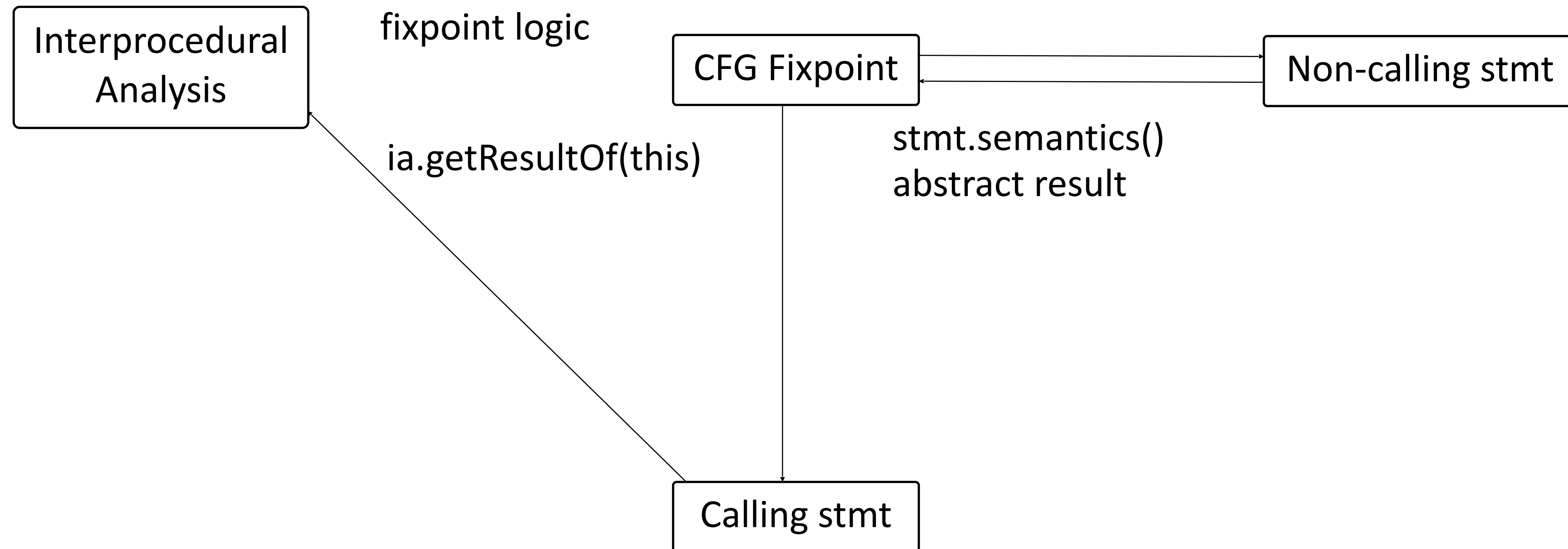
The Interprocedural Analysis



Interprocedural analysis can:

- analyze CFGs in isolation
- start from “main” and follow calls
- ...

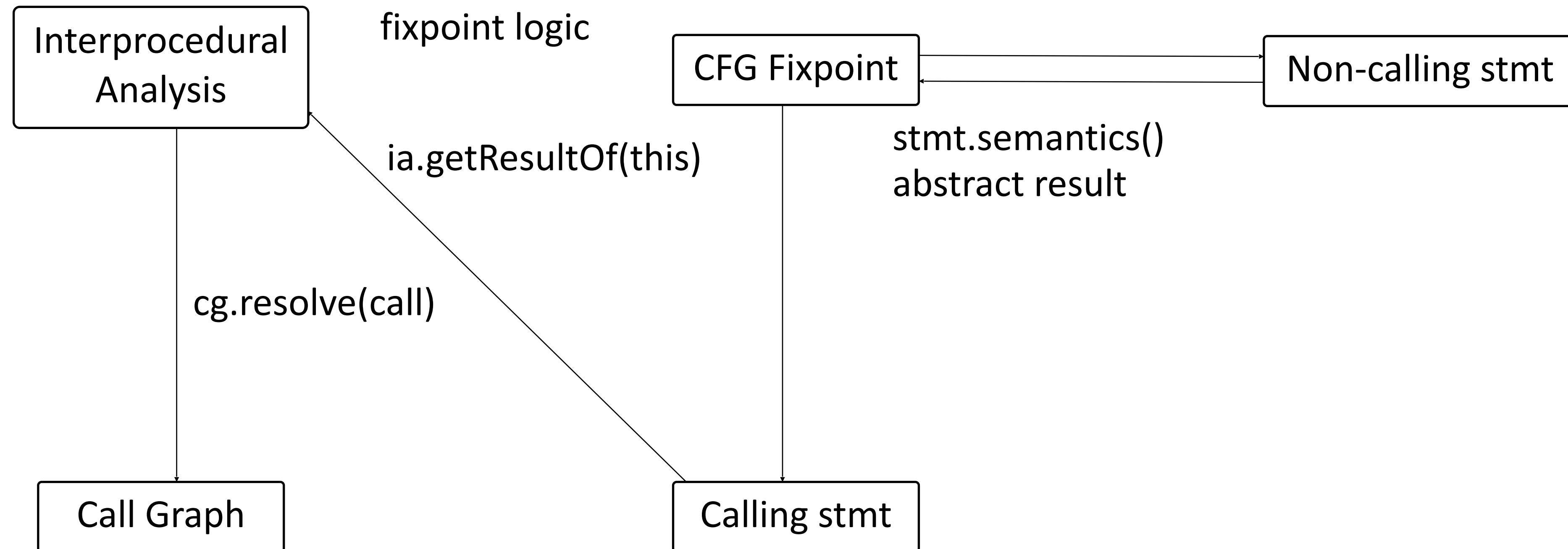
The Interprocedural Analysis



Interprocedural analysis can:

- analyze CFGs in isolation
- start from “main” and follow calls
- ...

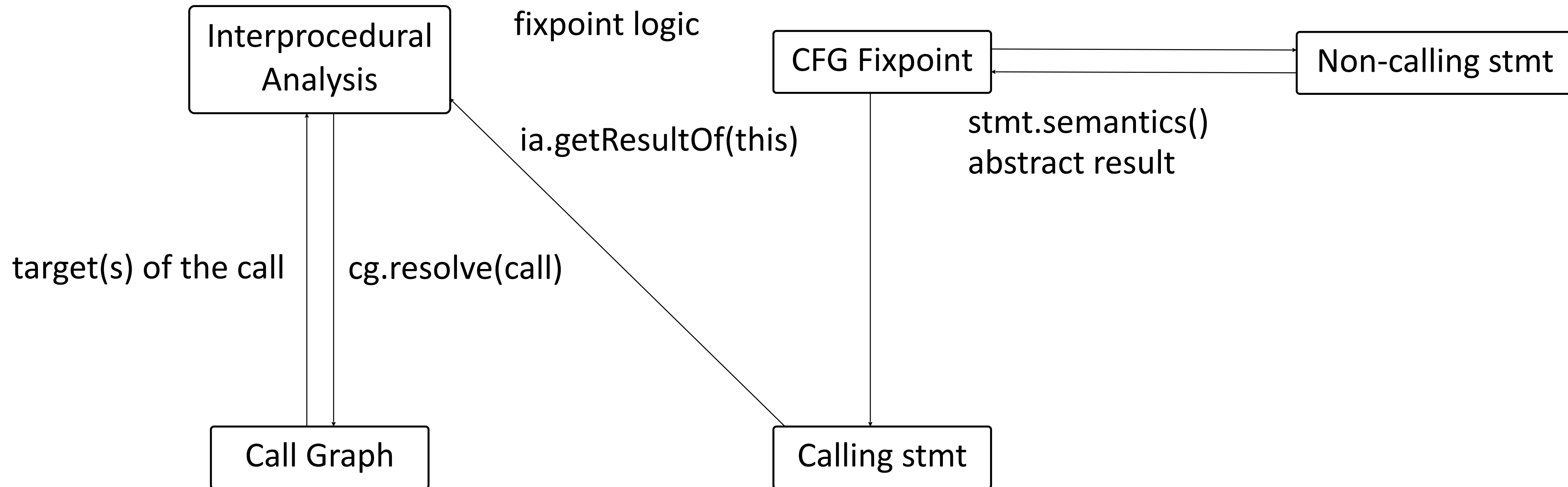
The Interprocedural Analysis



Interprocedural analysis can:

- analyze CFGs in isolation
- start from “main” and follow calls
- ...

The Interprocedural Analysis



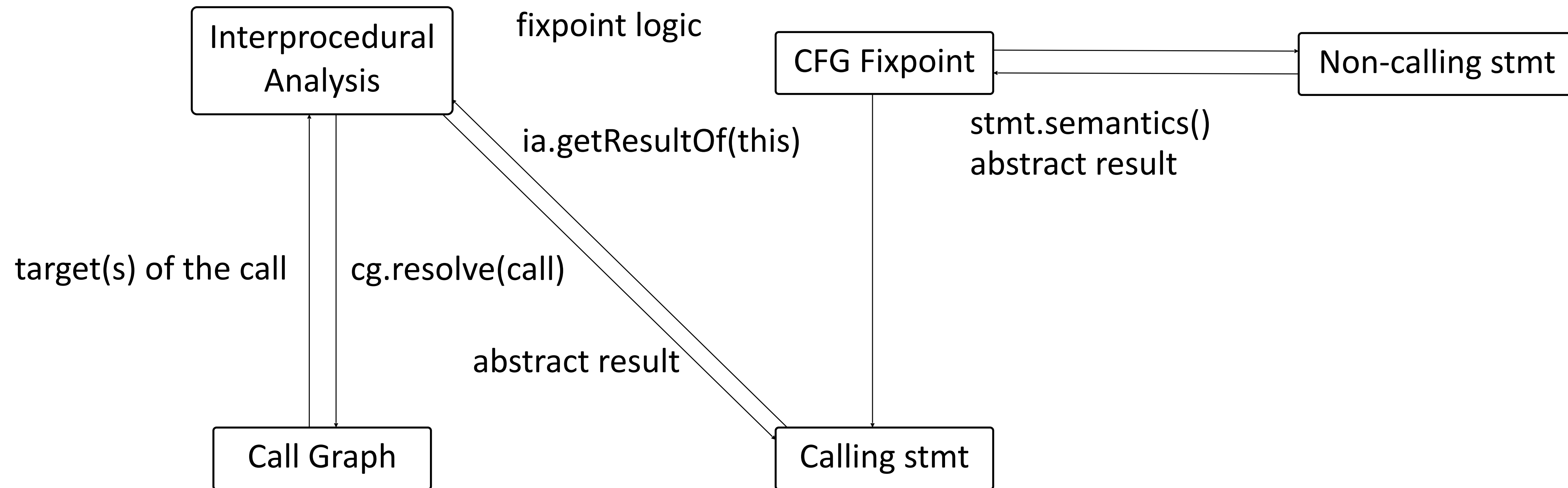
To find targets, call graph can use:

- runtime types of receiver
- all instances of static type of receiver
- ...

Interprocedural analysis can:

- analyze CFGs in isolation
- start from “main” and follow calls
- ...

The Interprocedural Analysis



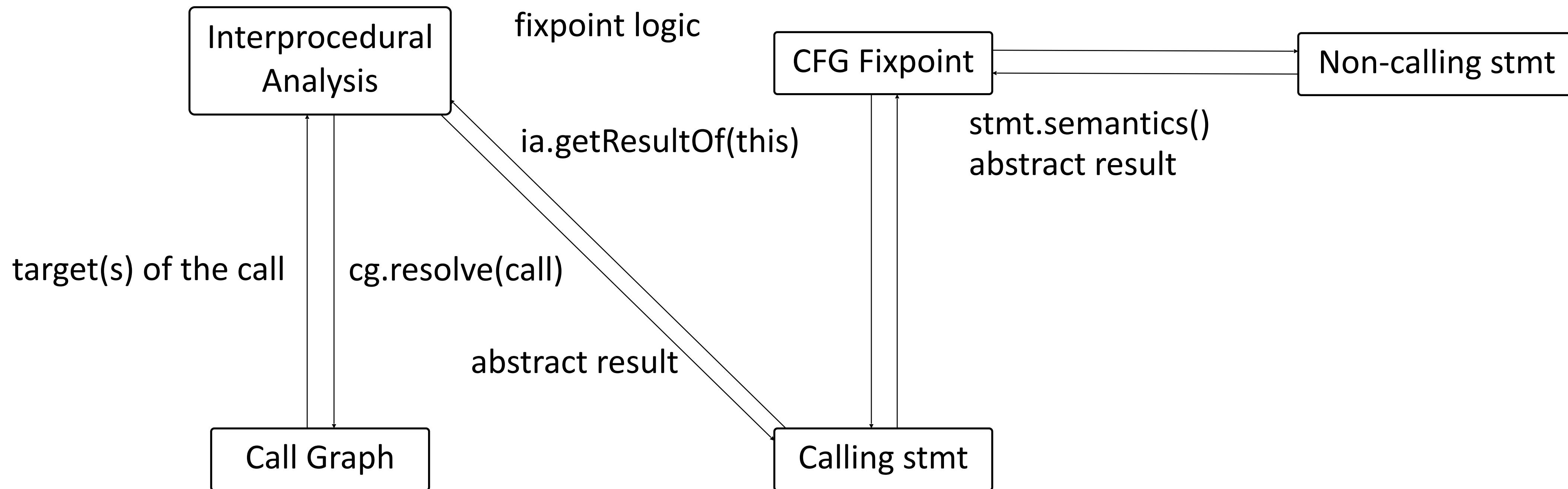
To find targets, call graph can use:

- runtime types of receiver
- all instances of static type of receiver
- ...

Interprocedural analysis can:

- analyze CFGs in isolation
- start from “main” and follow calls
- ...

The Interprocedural Analysis



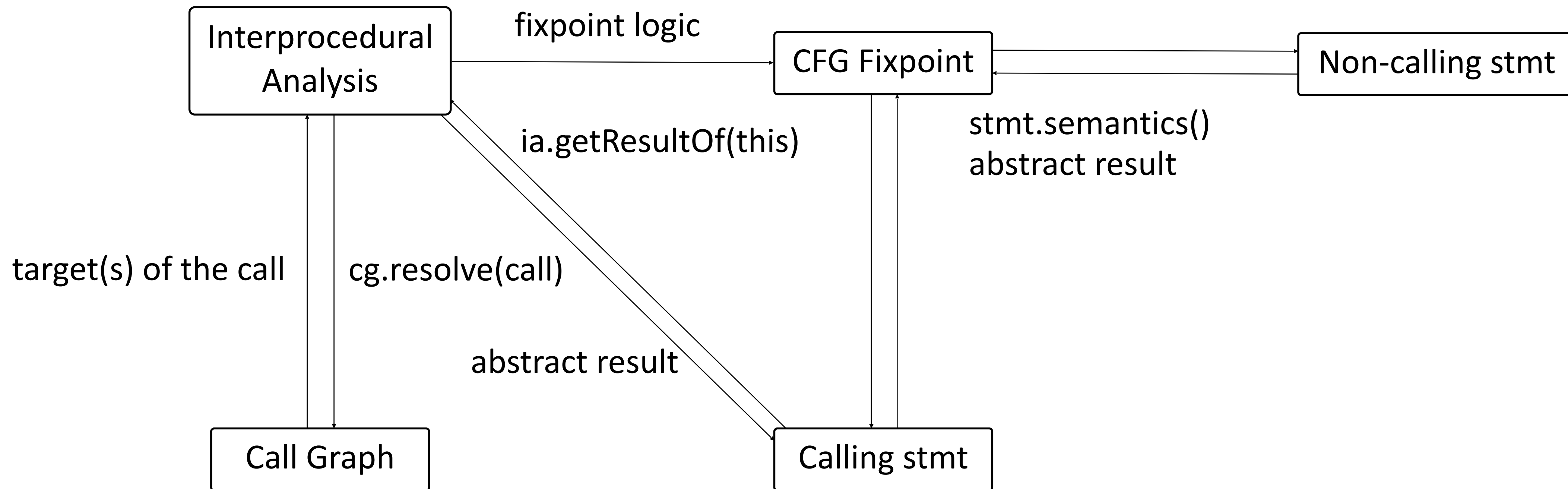
To find targets, call graph can use:

- runtime types of receiver
- all instances of static type of receiver
- ...

Interprocedural analysis can:

- analyze CFGs in isolation
- start from “main” and follow calls
- ...

The Interprocedural Analysis



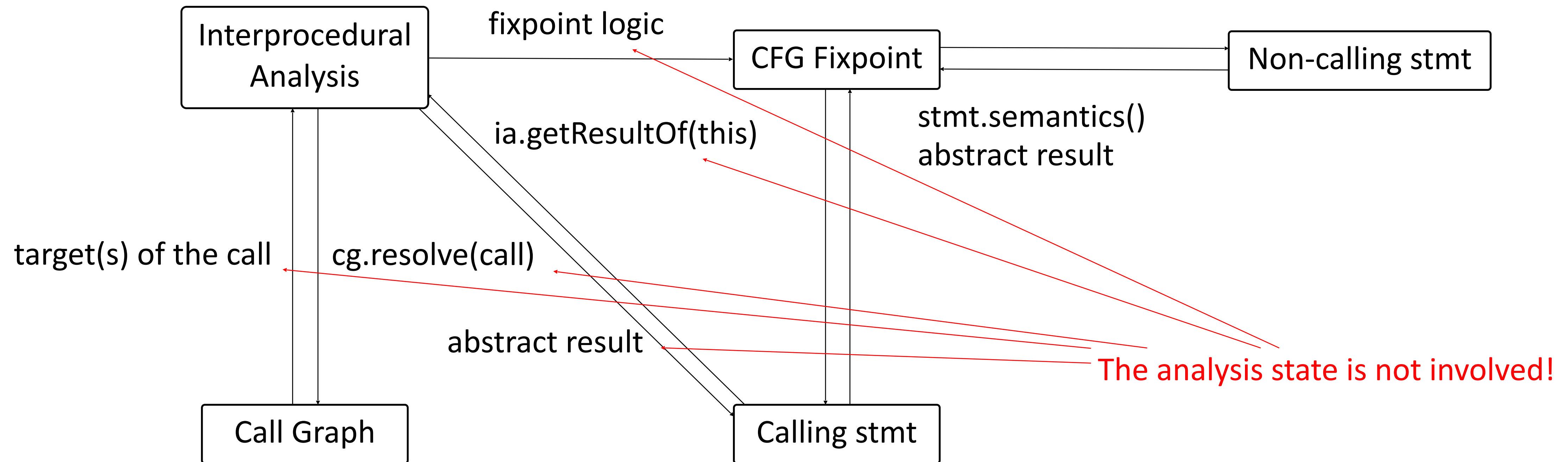
To find targets, call graph can use:

- runtime types of receiver
- all instances of static type of receiver
- ...

Interprocedural analysis can:

- analyze CFGs in isolation
- start from “main” and follow calls
- ...

The Interprocedural Analysis



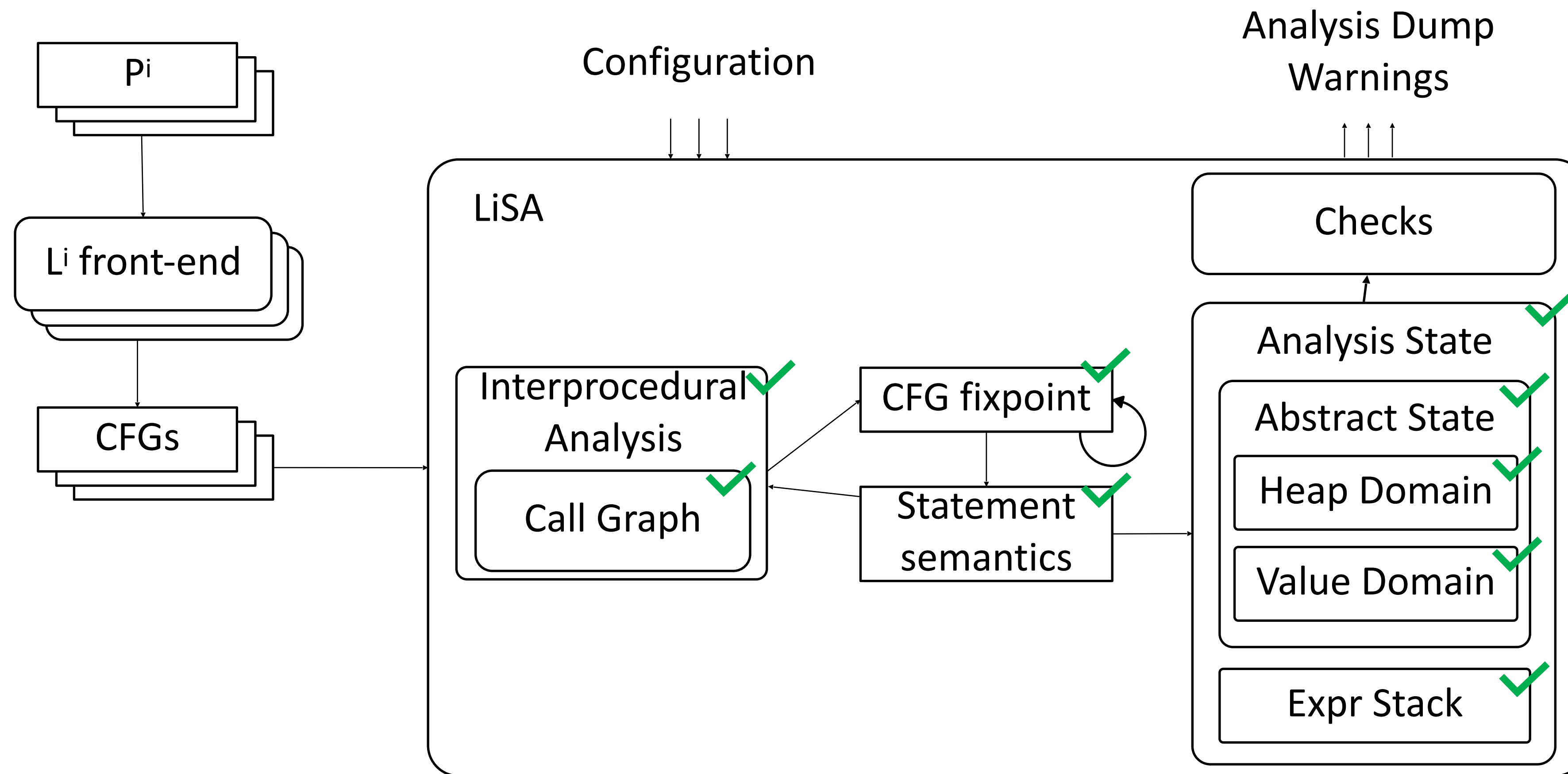
To find targets, call graph can use:

- runtime types of receiver
- all instances of static type of receiver
- ...

Interprocedural analysis can:

- analyze CFGs in isolation
- start from “main” and follow calls
- ...

LiSA overview



Recap: what does LiSA analyze?

- LiSA program is made up of CFGs (+ extra information)
- CFGs encode flow control in their structure
- CFG nodes are language-specific

Syntax

- CFG nodes rewrite into symbolic expressions

Semantics

- Calls + orchestration handled by Interprocedural Analysis + Call Graph
- Memory is modeled by the Heap Domain
- Values are modeled by the Value Domain
- All components are independent from one another

Analysis

Recap: what does LiSA analyze?

- LiSA program is made up of CFGs (+ extra information)
- CFGs encode flow control in their structure
- CFG nodes are language-specific
- CFG nodes rewrite into symbolic expressions
- Calls + orchestration handled by Interprocedural Analysis + Call Graph
- Memory is modeled by the Heap Domain
- Values are modeled by the Value Domain **Focus for today!**
- All components are independent from one another

Relational vs Non-Relational

Abstract domains can be:

- Non relational: track values of each variable independently
 - Constant Propagation: $x \rightarrow 3$
 - Intervals: $x \rightarrow [2, 10]$

Relational vs Non-Relational

Abstract domains can be:

- Non relational: track values of each variable independently
 - Constant Propagation: $x \rightarrow 3$
 - Intervals: $x \rightarrow [2, 10]$
- Relational: track relations between variables, and optionally their value
 - Upper bounds: $x > y$
 - Linear inequalities: $x + y > 10, z < 0$

Relational vs Non-Relational

Abstract domains can be:

- Non relational: track values of each variable independently
 - Constant Propagation: $x \rightarrow 3$
 - Intervals: $x \rightarrow [2, 10]$
- Relational: track relations between variables, and optionally their value
 - Upper bounds: $x > y$
 - Linear inequalities: $x + y > 10, z < 0$

Can be really complex!
(later in the course)

Non-relational domains

- Track values of single variables through a map
- Map changes only with assignments
- Evaluation logic is domain-specific

Non-relational domains

- Track values of single variables through a map
- Map changes only with assignments
- Evaluation logic is domain-specific

Non-relational domains

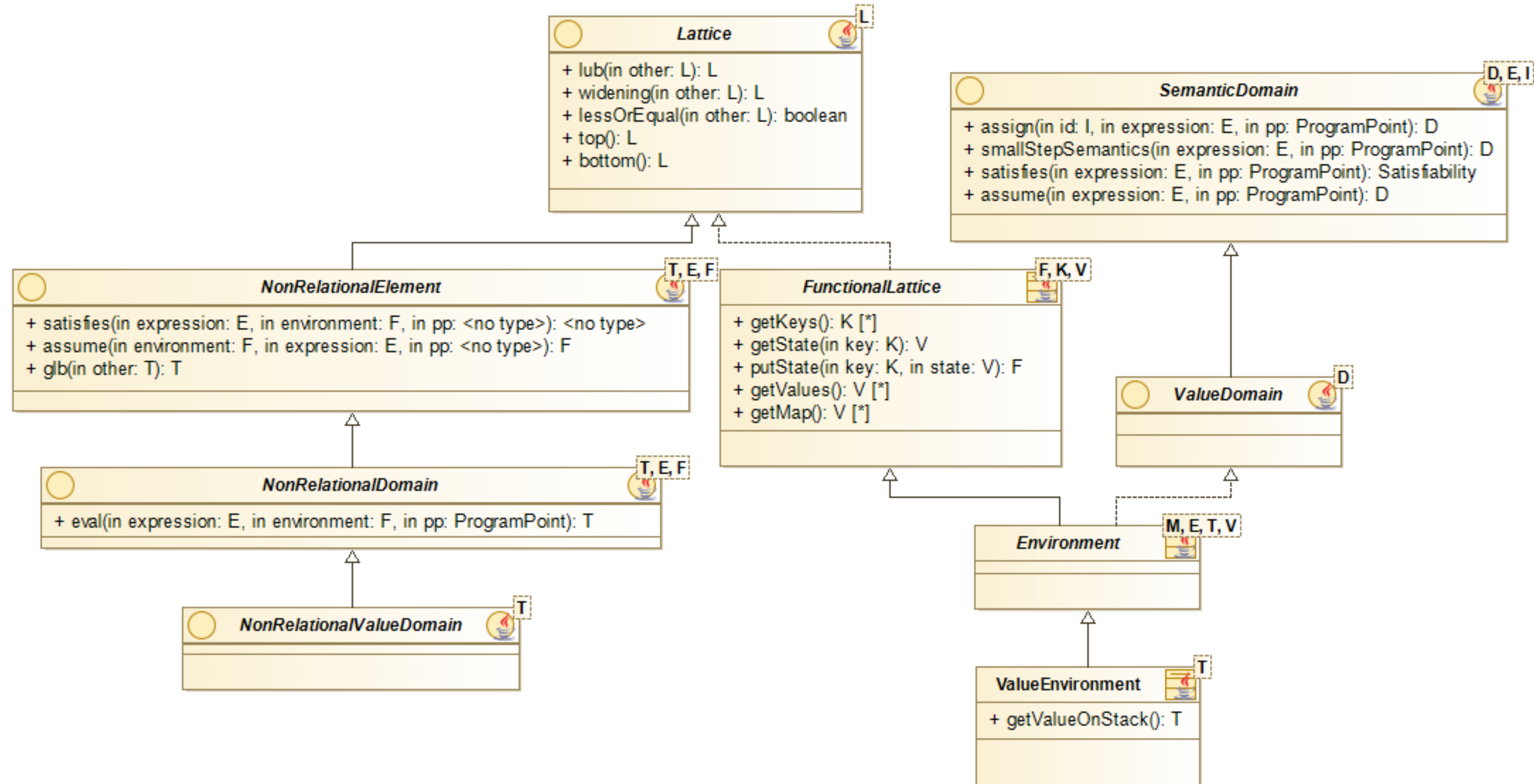
- Track values of single variables through a map
- Map changes only with assignments
- Evaluation logic is domain-specific

Similar to dataflow analyses!

Same strategy:

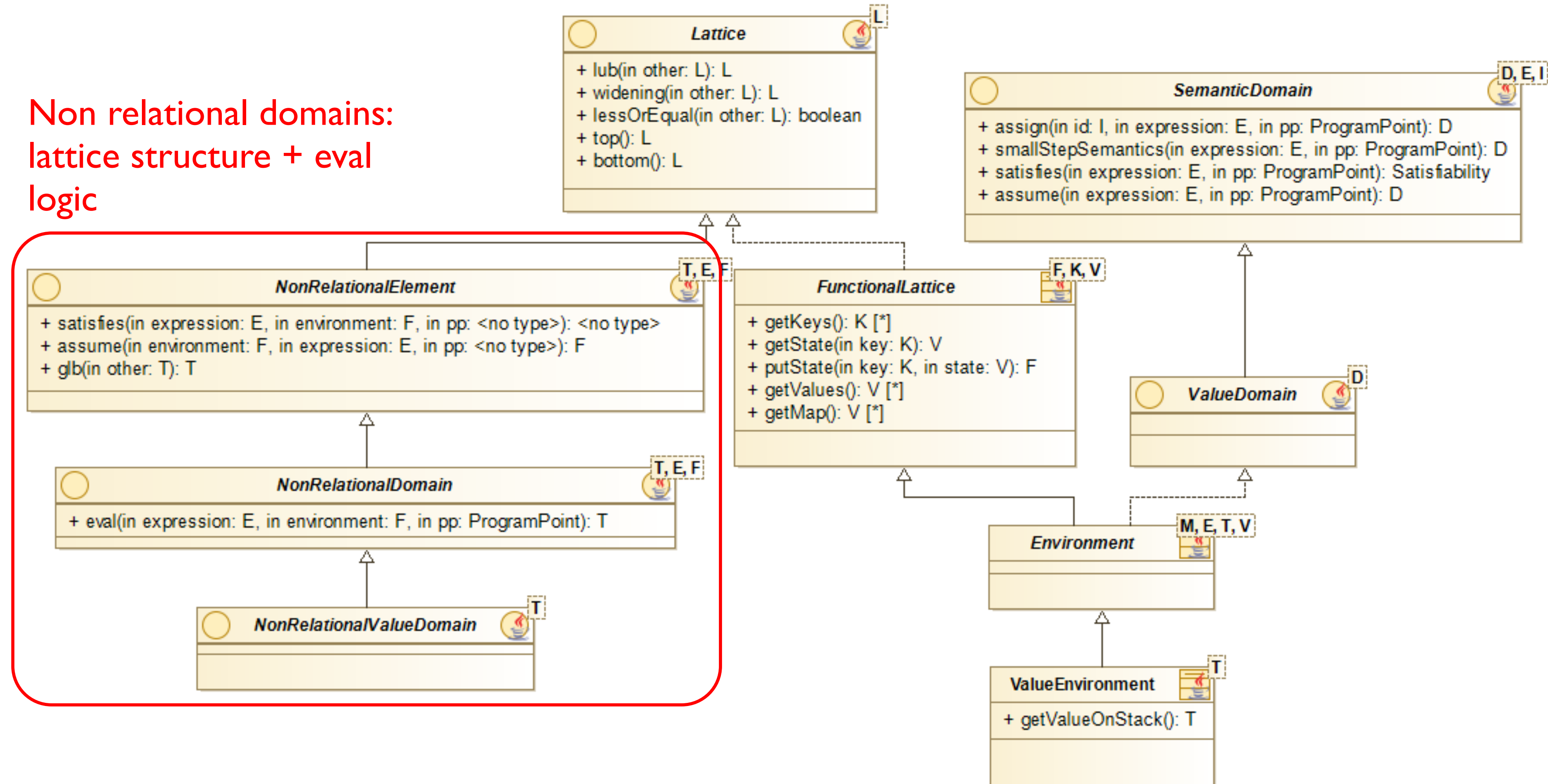
- Abstract common logic (map and assignment)
- Build instances of atomic information
- Instances contain the logic for expression evaluation

Non-relational classes

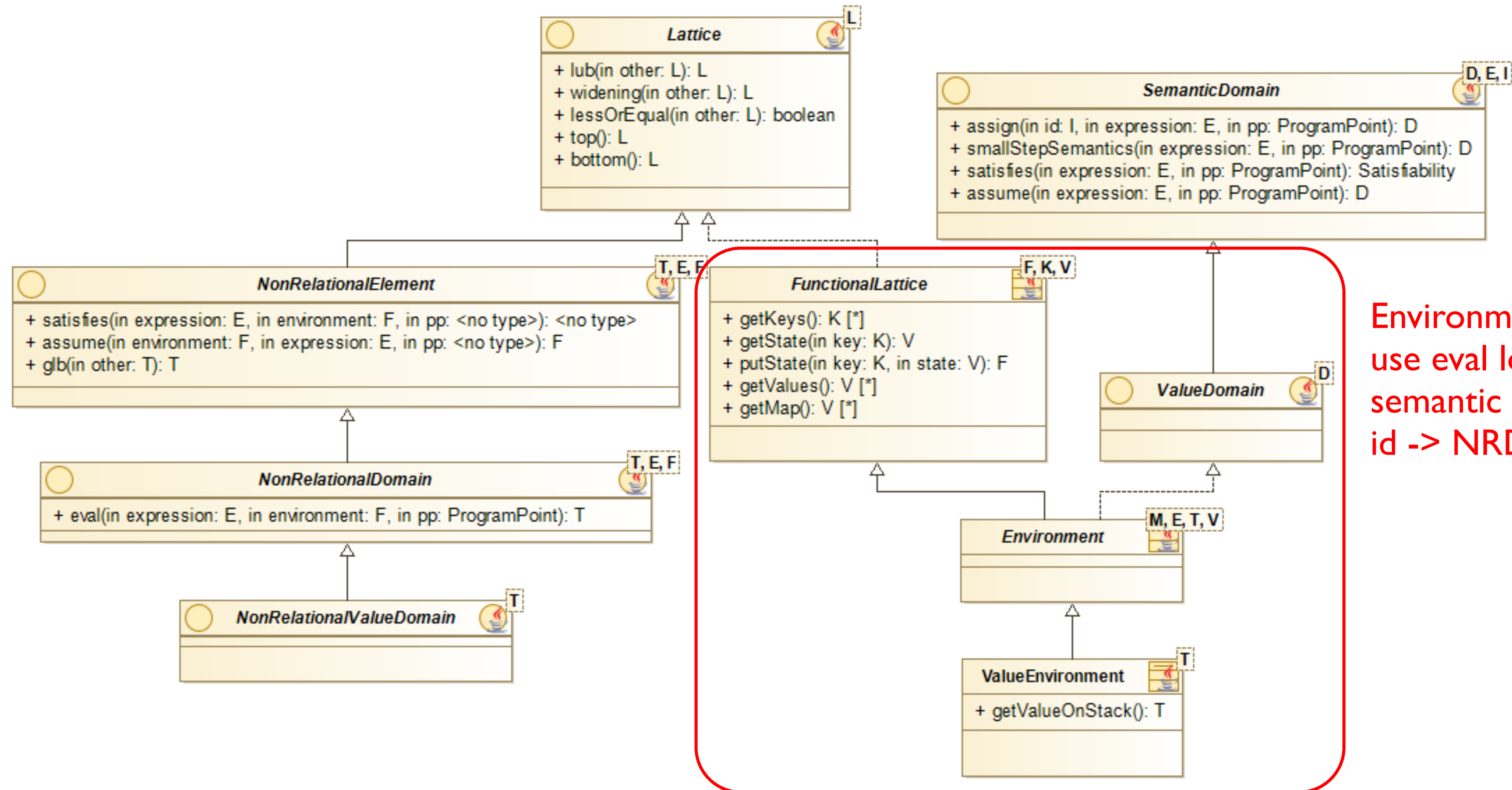


Non-relational classes

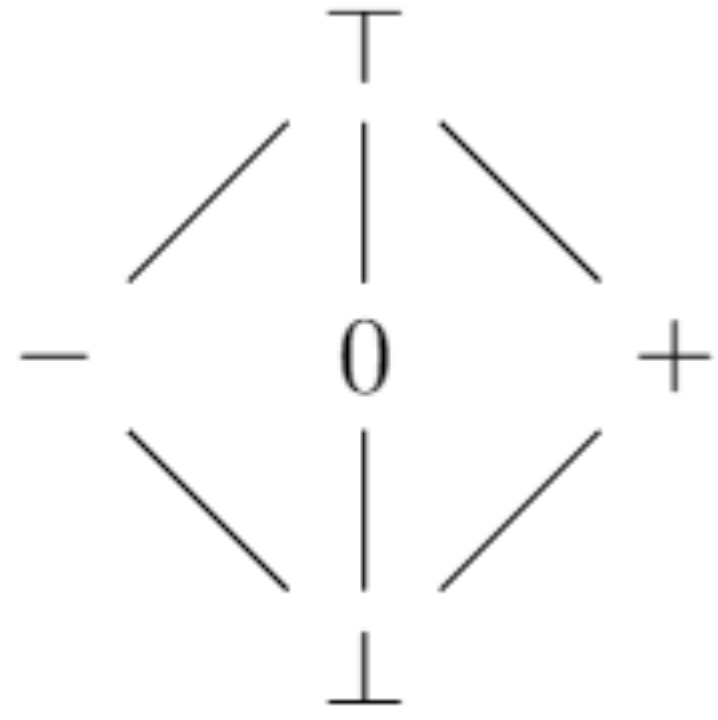
Non relational domains:
lattice structure + eval
logic



Non-relational classes



Environments:
use eval logic for
semantic operations +
id -> NRD



Sign Domain

Coding demo